A Polymorphic Moded Type System for Grassroots Logic Programs, by AI, for AI

Anonymous Author(s)

Anonymous Institution

Abstract. Grassroots Logic Programs (GLP) is a concurrent logic programming language in which logic variables are partitioned into paired readers and writers. An assignment is produced at most once via a writer and consumed at most once via its paired reader, and may contain additional readers and/or writers. This enables the concise expression of rich multidirectional communication modalities.

Extending "Logic Programs as Types for Logic Programs" (LICS'91), we develop a *moded-atom semantics*, in which each atom subterm carries a mode—consumed or produced—and partial computations, including those that deadlock, fail or never terminate, contribute moded atoms too; types are regular sets of *moded paths* over it. We define GLP well-typing syntactically and prove it sound: every output path of a well-typed program conforms to its declared type; dually, every input path a declaration admits is accepted by some clause. Subtyping, covariant in outputs and contravariant at mode inversions as in session types, preserves soundness. A parameterised procedure is checked at each instantiation or, if it inspects none of its parameters, once against opaque types. Modules are checked separately and linked by compilation, and a module arriving at run time is activated on equality of type identities, its own types opaque to its activator.

A type checker for GLP was implemented *by* AI (Claude) in Dart from the mathematical specification of this paper. The motivation for typing GLP was *for* AI: the human designer and AI first agree on the types, the procedure declarations and the intent of the procedures; AI then writes, tests and debugs the GLP code, the type checker catching the mode errors that had required tedious joint human-AI debugging, and the human oversees the design rather than the code.

1 Introduction

Grassroots Logic Programs. Grassroots Logic Programs (GLP) is a concurrent logic programming language in which logic variables are partitioned into paired *readers* and *writers*, recalling both linear logic [18] and futures/promises [2,14]: an assignment is produced at most once via the sole occurrence of a writer (promise) and consumed at most once via the sole occurrence of its paired reader (future), and may contain additional readers and/or writers.

Hence, GLP provides for the concise expression of multidirectional communication modalities, including bidirectional communication and the dynamic expansion and reconfiguration of communication networks. GLP's design eschews unification in favour of simple term matching. Being relational and nondeterministic, GLP—like its predecessor concurrent logic languages [39]—can express nondeterministic asynchronous concurrent processes such as the fair merging of streams. GLP is designed for the implementation of smartphone-based, serverless, grassroots platforms [41]. A digital platform is *grassroots* [40,42] if it can have multiple instances that can (*i*) operate independently of each other and of any global resource other than the network, and (*ii*) coalesce into ever larger instances, possibly resulting in a single global instance. The SRSW restriction makes GLP suitable for grassroots distributed execution: each variable assignment has exactly one producer and one consumer, so assignments can be communicated between independent agents without consensus.

Typing GLP. "Logic Programs as Types for Logic Programs" [15] defined types as regular sets of paths over the Herbrand atom semantics of a logic program. In this paper, we develop a *moded-atom semantics* that extends the standard Herbrand atom semantics in two ways: (*i*) each atom subterm carries a *mode*, recording whether it is consumed from or produced to the environment; and (*ii*) partial computations, including those that deadlock, fail, or never terminate, also contribute moded atoms to the semantics. We define types to be regular sets of *moded paths* over this semantics, define GLP well-typing syntactically, and prove that a well-typed program is sound: every output path in its well-typed moded-atom semantics conforms to its declared output type.

Types are specified using BNF rules with a complementation operator ? that relates producer and consumer views: a type `Stream` implicitly defines its dual `Stream?`, with modes complemented throughout. Procedure type declarations specify the mode of each argument, e.g., `merge(Stream?,Stream?,Stream)` declares that `merge` consumes two streams and produces one.

A GLP program is *well-typed* if three conditions hold. First, *output conformance*: each clause's head and body must have paths accepted by the declared types. Second, *input coverage*: every input path permitted by the type must be accepted by some clause. Third, *variable-pair compatibility*: a writer X and its paired reader $X?$ have dual types, relaxed in Section 4.4 to the type of X being a *subtype* of the dual of the type of $X?$ [17].

The *well-typed moded-atom semantics* of a program is the set of moded atoms its runs produce from *well-typed initial goals*—goals whose arguments conform to the declared types; it makes no claim about runs from ill-typed goals. The soundness of well-typing over this semantics is the *covariant*, or subsumption, direction [32,8].

Programming with AI. AI is used here in two distinct disciplines: *implementing* the GLP type checker, and *programming* in GLP.

A type checker for Typed GLP was implemented in Dart [19] *by* AI (Claude) via a three-tier methodology: (*i*) the mathematical specification of Typed GLP (this paper); (*ii*) an English+pseudocode spec, derived from (*i*) by AI;

(*iii*) Dart code, derived from (*ii*) by AI. The implementation performs type AST and DFA construction, well-typed term and clause checking, subtyping, parameterised-type expansion with parameter-instantiation inference, and construction of moded heads from declarations; it is validated by positive and negative type-checking tests, SRSW-violation tests, and invalid-guard tests over a corpus of typed example programs. The methodology is iterative in both directions: gaps found during code generation drove revisions to the specification and, in turn, to the definitions of this paper.

This is the sense in which moded types are *for* AI: they are a specification language for data types and communication modalities, providing a precise interface between the human designer and the AI programmer [13,30,4]. The discipline we developed with Typed GLP is for the human designer and AI to jointly develop the formal type definitions and declarations together with the informal intent of the declared procedures, and only then let AI write the GLP code. Mode errors—where a writer and a reader are confused—are a characteristic class of error in GLP; untyped, they appear only at run time, as silent suspensions, and require expensive joint human-AI debugging to locate. Against a settled type system they are caught by the type checker at compile time, and AI corrects them from the diagnostic: for example, in a befriending program from the implementation corpus, a clause head was rejected at compile time—"Variable mode mismatch: reader requires ↓ (consume), got ↑ (produce); Path: (Decision?, 0, output)"—and corrected with no human debugging. Following the initial agreement on types, declarations, and intent, AI develops, type-checks, tests, and debugs the GLP code, with the human providing design-level oversight, including language-design decisions, rather than code-level intervention. The implementation, with its type-checking test corpus, is publicly available; the link is withheld for anonymity.

Experience. Since December 2025 the programs of the public GLP repository have been written in Typed GLP by AI. At the time of writing, the repository holds 131 programs outside its test corpus, 24 of them directory programs of several modules; 118 of them compile, which requires that they type-check, 20 of these directory programs. These 118 comprise 25,524 lines of GLP in 7,557 clauses and 3,593 procedures, with 4,146 procedure declarations and 940 type definitions. The test corpus holds a further 426 units in 5,327 lines, 238 of which compile; most of the other 188 are ill-typed or ill-formed on purpose, to verify that the type checker and the compiler reject them as such. Modes are used throughout: 136 of the type definitions carry a mode inversion, a position at which communication reverses direction, as where a request carries the variable for its reply. Polymorphism is used too: 111 of the declarations and 50 of the type definitions are parameterised, and their 629 call sites induce 425 instantiations, each of which would otherwise need a monomorphic copy of its declaration.

Related Work. Moded types extend the path-based LP type system of [15] and its polymorphic variant [54] with communication directionality, and are related to mode and directionality analysis of Prolog [6], logic programs [1,45], linear logic [34], and concurrent logic programming [50,48]. Unlike these, GLP's modes

arise from reader/writer pairing and are checked against regular sets of moded paths. To the best of our knowledge, moded types are the first type system for a concurrent logic language whose communication directionality is part of the type structure, obtained as an abstraction of a moded-atom semantics rather than from a separate mode analysis or inference. Path-based LP types rest on tuple-distributive sets [29,55], and Mycroft and O'Keefe's polymorphic type system for Prolog [31] is prescriptive. Ueda's moded Flat GHC [49] guarantees that each variable is written once, half of GLP's SRSW restriction.

Directionality is central to session types and their linear-logic reading [7,52,21], whose subtyping is covariant in outputs and contravariant in inputs [17], as is GLP's. GLP also sits within the broader concurrency tradition of concurrent constraint programming [36], the π-calculus [28], Janus [35], and interaction nets [23], and instantiates the convergence Tick [46] observed across concurrent logic programming languages. Concurrent logic languages [39] are GLP's operational ancestors. They include Concurrent Prolog [38], GHC [47], PARLOG [11] and Flat Concurrent Prolog [27], whose read-only unification is order-dependent [24]; SRSW resolves this, term matching sufficing in place of unification. Janus was implemented sequentially in Prolog [12]. Session types [20,51] type communication protocols, and session duality, which swaps input and output, corresponds to the dual of a moded type. CoLF_{ω_1} [10] interprets logic variables as channels, but its uniqueness requirement precludes fair merge; SILL_S [3] shares state by acquire and release, where GLP merges streams. The specification-first discipline of this paper combines Design by Contract [26], Literate Programming [22] and type-driven development [5]. A fuller treatment is in Appendix K.

Paper Structure. Section 2 presents GLP, its operational semantics, and the moded-atom semantics underlying the type system. Section 3 introduces moded types and procedure declarations. Section 4 defines GLP well-typing and its subtyping relaxation. Section 5 proves the soundness of well-typing. Section 6 adds parameterised types by expansion. The proofs of the propositions of Section 2 are in Appendix G; the other proofs are in place. Appendices A to F carry the material of the full version that the body omits.

2 Grassroots Logic Programs

Grassroots Logic Programs (GLP) extend LP by (*i*) adding a paired *reader* $X?$ to every "ordinary" logic variable X, now called a *writer*; (*ii*) restricting variables in goals and clauses to have at most a single occurrence (SO); and (*iii*) requiring that a variable occurs in a clause iff its paired variable also occurs in it (single-reader single-writer, SRSW). The result eschews unification in favour of simple term matching, is linear-logic-like [18], and is futures/promises-like [2,14]: each assignment $X := T$ is produced at most once via the sole occurrence of a writer (promise) X, and consumed at most once via the sole occurrence of its paired reader (future) $X?$.

2.1 Syntax

Definition 21 (GLP Variables). *Let $\mathcal{V}$ be the set of LP variables, henceforth called **writers**. Define $\mathcal{V}? = \{X? \mid X \in \mathcal{V}\}$, called **readers**. The set of all GLP variables is $\hat{\mathcal{V}} = \mathcal{V} \cup \mathcal{V}?$. A writer X and its reader $X?$ form a **variable pair**.*

GLP terms, unit goals, goals, and clauses are as in LP (Definition A1), but defined over the variables in $\hat{\mathcal{V}}$.

Definition 22 (Single-Occurrence (SO) Invariant). *A term, goal, or clause satisfies the **single-occurrence (SO) invariant** if every variable occurs in it at most once.*

Definition 23 (GLP Program). *A clause C satisfies the **single-reader/single-writer (SRSW) restriction** if it satisfies SO and a variable occurs in C iff its paired variable also occurs in C. A **GLP program** is a finite sequence of clauses satisfying SRSW; clauses for the same predicate form a **procedure**. Let $\hat{\mathcal{G}}(P)$ denote the set of goals over $\hat{\mathcal{V}}$ and the vocabulary of P that satisfy SO.*

As we shall see (Proposition 211), reducing a goal satisfying SO with a clause satisfying SRSW results in a goal satisfying SO. The purpose of the SRSW restriction is to prevent multiple writer occurrences racing to assign a variable.

Example 24 (Fair Merge) *Consider the standard concurrent logic program for fairly merging two streams, written in GLP:*

```
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge(Xs, [], Xs?).
merge([], Ys, Ys?).
```

and the goal `merge([1,2,3|Xs?],[a,b|Ys?],Zs)`. *Both the goal and each clause satisfy SO, and each clause satisfies SRSW. The first two clauses swap inputs in recursive calls, ensuring fairness when both streams are available.*

2.2 Operational Semantics

As is standard for concurrent programming languages [33], we provide GLP with operational semantics using a nondeterministic transition system, termed *cGLP* for concurrent GLP; the paper defining GLP [41] also defines a multiagent operational semantics for it. The basic Reduce transition of cGLP (Definition 29) differs from LP in (*i*) using a writer mgu instead of a regular mgu—the fundamental use of GLP readers for synchronisation—and (*ii*) choosing the first applicable clause instead of any clause to enable the expression of fair concurrent programs such as fair merge. Communicate propagates an assignment from a writer to its paired reader.

Definition 25 (Writers Substitution, Assignment, Readers Substitution and Counterpart). *A GLP **writer assignment** is a term of the form $X := T$, $X \in \mathcal{V}$, $T \notin \mathcal{V}$, satisfying SO. Similarly, a GLP **reader assignment***

is a term of the form $X? := T$, $X? \in \mathcal{V}?$, $T \notin \mathcal{V}$, *satisfying SO. A* ***writers (readers) substitution*** σ *is the substitution implied by a set of writer (reader) assignments that jointly satisfy SO. Given a writer assignment* $X := T$, *its* ***readers counterpart*** *is* $X? := T$, *and given a writers substitution* σ, *its* ***readers counterpart*** $\sigma?$ *is the readers substitution defined by* $X?\sigma? = X\sigma$. *Dually, given a reader assignment* $X? := T$, *its* ***writers counterpart*** *is* $X := T$, *and given a readers substitution* τ, *its* ***writers counterpart*** $\tau!$ *is the writers substitution defined by* $X\tau! = X?\tau$. *The* ***pair completion*** *of a readers substitution* σ *is* $\sigma^\star = \sigma \cup \sigma!$, *applied to a fixed point.*

Definition 26 (GLP Renaming, Renaming Apart). *A* ***GLP renaming*** *is an injective substitution* $\rho : \hat{\mathcal{V}} \to \hat{\mathcal{V}}$ *such that for each* $X \in \mathcal{V}$: $X\rho \in \mathcal{V}$ *and* $X?\rho = (X\rho)?$. *Two GLP terms* ***have a variable in common*** *if for some writer* $X \in \mathcal{V}$, *either* X *or* $X?$ *occurs in both. A GLP renaming* ρ ***renames*** T' ***apart from*** T *if* $T'\rho$ *and* T *have no variable in common.*

Definition 27 (Writer MGU [41]). *Given two GLP unit goals* A *and* H, *a* ***writer mgu*** *is a writers substitution* σ *such that* $A\sigma = H\sigma$ *and* σ *is most general among such substitutions.*

Definition 28 (GLP Goal/Clause Reduction). *Given GLP unit goal* A *and clause* C, *with* H `:-` B *being the result of the GLP renaming of* C *apart from* A, *the* ***GLP reduction*** *of* A *with* C ***succeeds with result*** (B, σ) *if* A *and* H *have a writer mgu* σ.

Definition 29 (cGLP Transition System [41]). *Given a GLP program* P, *an* ***asynchronous resolvent*** *over* P *is a pair* (G, σ) *where* $G \in \hat{\mathcal{G}}(P)$ *and* σ *is a readers substitution.*

A transition system $cGLP = (\mathcal{C}, c_0, \mathcal{T})$ *is a* ***cGLP transition system*** *over* P *and initial goal* G_0 *satisfying SO if:*

1. $\mathcal{C}$ *is the set of all asynchronous resolvents over* P
2. $c_0 = (G_0, \emptyset)$
3. $\mathcal{T}$ *is the set of all transitions* $(G, \sigma) \to (G', \sigma')$ *satisfying either:*
   (a) ***Reduce:*** *there exists unit goal* $A \in G$ *such that* $C \in P$ *is the first clause for which the GLP reduction of* A *with* C *succeeds with result* $(B, \hat{\sigma})$, $G' = (G \setminus \{A\} \cup B)\hat{\sigma}$, *and* $\sigma' = \sigma \circ \hat{\sigma}?$
   (b) ***Communicate:*** $\{X? := T\} \in \sigma$, $X? \in G$, $G' = G\{X? := T\}$, *and* $\sigma' = \sigma$

Definition 210 (Run, Outcome). *A* ***run*** *of cGLP is a computation* $(G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ *starting from* c_0, ***complete*** *if it is infinite or ends in a* ***terminal*** *configuration, one from which no transition is enabled. The* ***variables of a configuration*** (G, σ) *are those of* G *and of* σ, *its readers and the terms assigned to them. A run is* ***proper*** *if for any* $1 \leq i < n$, *a variable of* (G_{i+1}, σ_{i+1}) *that is not one of* (G_i, σ_i) *is not one of any* (G_j, σ_j), $j < i$; *the* ***outcome*** *of a proper run is* $(G_0$ `:-` $G_n)\sigma_n^\star$, *and the run is* ***successful*** *if* $G_n = \emptyset$.

GLP runs satisfy two safety properties.

Proposition 211 (SO Preservation) *If the initial goal G_0 satisfies SO, then every goal in a proper cGLP run satisfies SO.*

Proposition 212 (Monotonicity) *In any proper cGLP run, if unit goal A can reduce with clause C at step i, then either an instance of A has been reduced by step $j > i$, or an instance of A can still reduce with C at step j.*

2.3 Term Matching Eschews Unification

If two terms T_1 and T_2 that jointly satisfy SO are unifiable with an mgu σ, then σ maps any variable in T_1 to a subterm of T_2 and vice versa. Hence, the SO invariant of GLP allows eschewing unification in favour of *term matching* that performs joint term-tree traversal and collects variable assignments along the way, as follows.

Definition 213 (Term Matching). *Given a goal T_1 and a renamed-apart clause head T_2 that jointly satisfy SO, their* ***term matching*** *proceeds via the joint traversal of the term-trees of T_1 and T_2, consulting the following table at each pair of joint vertices (T_1 ranging over the goal's subterms, T_2 over the head's), where X_1, X_2 denote writers, $X_1?, X_2?$ denote readers, and f/n denotes a non-variable term, a constant when $n = 0$ and a compound term when $n > 0$:*

$T_1 \backslash T_2$	*Writer* X_2	*Reader* $X_2?$	*Term* f_2/n_2
Writer X_1	*fail*	$X_1 := X_2?$	$X_1 := T_2$
Reader $X_1?$	$X_2 := X_1?$	*fail*	*suspend on* $X_1?$
Term f_1/n_1	$X_2 := T_1$	*fail*	*fail if* $f_1 \neq f_2$ *or* $n_1 \neq n_2$

The writer mgu is the union of all writer assignments if no fail *was encountered and the suspension set is empty.*

2.4 Moded-Atom Semantics

The basic semantic object of a GLP program's behaviour is the moded term, which records whether each subterm was produced or consumed during the computation.

Definition 214 (Moded Term, Dual). *Given a GLP term T, a* ***moded term*** *T' corresponding to T is the result of adding one of two* ***mode annotations****,* ***consume*** *↓ or* ***produce*** *↑, to T and to every non-variable subterm of T. Given a moded term T, its* ***dual*** *$T?$ is obtained by* complementation*: complementing every mode annotation and replacing every variable by its paired variable.*

By convention, a writer carries the produce mode ↑ and its paired reader the consume mode ↓.

For example, consider the term:

```
merge([3|Xs?],Ys?,[3|Zs])
```

and a corresponding moded term:

`↓merge(↓[↓3|Xs?],Ys?,↑[↑3|Zs])`
Its dual is:
`↑merge(↑[↑3|Xs],Ys,↓[↓3|Zs?])`

Substitution extends to moded terms: for a moded term T and substitution σ, the moded term $T\sigma$ is obtained by replacing each variable X or $X? \in T \cap \operatorname{dom}\sigma$ with the term $X\sigma$ or $X?\sigma$, which inherits the mode of its position.

Definition 215 (Moded-atoms outcome and moded resolvent of a run). *Given a proper cGLP run of a GLP program $P = (Cs, D)$ with outcome $(G_0$ `:-` $G)\sigma^\star$ (Definition 210), let G'_0 be $G_0\sigma^\star$ moded per D: each consumed position carries mode ↓ and each produced position mode ↑. The **moded-atoms outcome** is the set of unit goals of G'_0; the **moded resolvent** G' is $G\sigma^\star$ moded the same way.*

Definition 216 (Moded-Atom Semantics). *The **moded-atom semantics** of a GLP program P is the set of all moded-atoms outcomes of every finite proper run of P with an SO initial goal.*

The full language also includes guards, the type-handling of which is specified in Appendix B.1.

3 Typed GLP

While GLP is untyped, we introduce Typed GLP; well-typing follows in Section 4.

3.1 Type Syntax

Type Declarations

LP types are concerned with the values logic variables may take. A GLP type also indicates whether a term, or a subterm, is produced or consumed. By convention, we define types from the perspective of their producer, which implicitly defines its dual—the same type but from the perspective of its consumer.

GLP types are specified using BNF rules with the complementation operator ?. For example, the stream producer type, of streams of elements of type `X` (a type parameter; Section 6):
`Stream(X) ::= [] ; [X|Stream(X)].`
implicitly defines the dual stream consumer type `Stream(X)?`. The primitive types `_` and `_?` are wildcards accepting any produced or consumed subtree, respectively.

A program declares every procedure it defines, and writes no `_` or `_?` in a declaration or in a type definition; the two wildcards stand only in a system builtin of `self.glp` and in a catalogue signature of the GLP paper that takes any term, where the runtime accepts any term.

A type that is not complemented (for example `Stream`) is an *output type*, and its dual (for example `Stream?`) is an *input type*. A type that contains both produced and consumed positions is an *interactive type*. An alternative in a type definition may also be a type name S, providing type union for types with disjoint top-level functors. This disjointness is required so that a term's top

functor selects its alternative deterministically. Two alternatives with the same functor are not allowed; a stream of integers interspersed with `switch` markers cannot be written as

```
Items ::= [] ; [Integer|Items] ; [switch|Items].
```

whose last two alternatives share `cons`. Factoring the element choice into its own type removes the clash:

```
Items ::= [] ; [Item|Items].
Item  ::= Integer ; switch.
```

The primitive types are `Integer`, `Real`, `String`, `Module` and `MutualRef`, the last an opaque handle on the tail of a stream, through which `mwm/2` appends in constant time. `Number`, `Constant`, and `Exp` (arithmetic expressions) are defined in the root `self.glp` (Appendix H): `Number ::= Integer ; Real` and `Constant ::= Number ; String ; Module`. `MutualRef` is not among them: a mutual reference holds the writer of a stream tail, so it is neither ground nor a constant type (Section 3.2).

The built-in type `Term` is the type of every produced term: it is defined as by a rule `Term ::= Integer ; Real ; String ; Module ;` f(`Term`, ..., `Term`), with an alternative for every functor f of every arity $n \geq 1$. Its dual `Term?` is the type of every consumed term. A program that takes terms of any functor as its data, as a compiler takes a source program held as a term, declares them `Term?`. Below a wildcard a term is not examined; below `Term` every position has the mode of `Term`, and the term is checked in full.

The formal specification of the type automaton induced by a typed GLP program appears in Section 4.2.

Procedure Declarations

Typed GLP includes a type declaration for each procedure. For example, the `merge` program could include the procedure declaration:

```
procedure merge(Stream?,Stream?,Stream).
```

which is a syntactic shorthand for defining a type named `Procedure` using the standard BNF syntax:

```
Procedure ::= merge(Stream?,Stream?,Stream).
```

Beyond syntactic convenience, the procedure declaration specifies the mode of each argument, essential for the well-typing conditions of Section 4.

3.2 Program Syntax

SRSW Relaxations.

The SRSW restriction (Definition 23) may be relaxed under certain conditions specified in Appendix B.1 and Appendix C.

Typed GLP Programs

Definition 31 (Typed GLP Program). *A **typed GLP program** $P = (Cs, D)$ consists of GLP clauses Cs and type declarations D such that:*

1. *Every procedure in Cs has exactly one type declaration in D.*
2. *Every procedure declared in D is defined by at least one clause in Cs, except a language primitive—a guard predicate or a body kernel—which the runtime implements (Appendix H.1).*
3. *Every type referenced in D is a primitive type or is defined by a type rule in D.*

4 GLP Well-Typing

4.1 Moded Paths

We type a moded term (Definition 214) by relating the paths of its term tree to those of a GLP type. A moded term can be equivalently represented as a *moded term-tree*, an LP term tree with edges labelled by (argument index, mode) pairs. For example, the subtree for the first argument is:

```
(0,↓): merge/3
    (1,↓): "."/2
        (1,↓): 3
        (2,↓): Xs?
```

with moded path `(0,↓) --> merge/3 --(1,↓)--> "."/2 --(1,↓)--> 3`.

Definition 41 (Moded Paths). *For a moded term T, paths(T) denotes the set of **moded paths** of its moded term tree. A path is an **input** (**output**) path according to the mode on the edge leaving its root—the first-argument edge: consume ↓ for an input path, produce ↑ for an output path. The mode on the root functor itself is not used for this classification. A moded term is **consumed** (**produced**) if all its mode annotations are consume (produce), and **I/O** if it is ↓-annotated with at most one mode-inversion to ↑ on any of its paths.*

A defined GLP type D also defines a regular set of moded paths, denoted *paths*(D), of the same form, ending at a primitive type or wildcard. The type names D passes through along a path are not themselves path positions. A moded term path, by contrast, ends at a primitive term—a constant or a variable.

Definition 42 (Consistent Paths). *Let x be a moded term path and y a GLP type path, with lengths $|x|$ and $|y|$ respectively. Let $k = \min(|x|, |y|)$. Then x and y are **consistent** if:*

1. *For all positions $i < k$ the functor/arity at position i are identical in x and y, and for all i with $1 \leq i < k$ the modes are identical; the mode at the root (position 0) is not compared.*
2. *The symbols at position k in x and y are **compatible** per the table below.*

#	**Term**	**Type**
1	f/n, mode m	f/n, mode m
2	`X?`	`_?`, or any symbol with mode ↓
3	`X`	`_`, or any symbol with mode ↑
4	integer	`Integer`, `Number`, or `_`
5	real	`Real`, `Number`, or `_`
6	string	`String` or `_`
7	constant c	c, `String`, or `_`
8	module term	`Module`
9	mutual reference term	`MutualRef`
10	any, mode ↓	`_?`
11	any, mode ↑	`_`

The last two rows apply when the type path ends at a wildcard; the remainder of the term path is not examined. The wildcard states `_` and `_?` subsume all primitive types of the matching mode. Rows 2 and 3 handle the dual case: the term path ends at a variable while the type path continues, and the remaining type structure is not examined.

Example 43 (Consistent Paths) *Consider the first* `merge` *clause with type* `merge(Stream?,Stream?,Stream)`*. For the first argument, the type path*
`(0,↓) --> merge --(1,↓)--> Stream? --(1,↓)--> "."/2 --(1,↓)--> _?`
and term path
`(0,↓) --> merge --(1,↓)--> "."/2 --(1,↓)--> X?` *are consistent: the paths are identical until position* $k = 4$*, where the term has reader* `X?` *and the type has* `_?`*, which are compatible.*

Definition 44 (Well-Typed Moded Term). *A moded term* T *is* ***well-typed*** *by a GLP type* D *if for each term path* $x \in paths(T)$ *there is a consistent path* $y \in paths(D)$ *and the types according to* D *of every variable pair in* T *are dual.*

Clause heads require special treatment. By rows 2 and 3 of the table, a variable is consistent only at a position of its own mode, a term at a position of either; so in a well-typed goal or body term a writer occurs at produced (↑) positions and a reader at consumed (↓) ones, and a *source clause head* inverts this: a head writer X at a ↓ position is an *input slot*, and a head reader `X?` at an ↑ position is an *output placeholder*. A head is moreover I/O-moded: mode inversions occur at most once per path. The moded head (Definition 45) restores the goal convention by complementing every head variable (step 2), so a reader is at every ↓ and a writer at every ↑—the form Definition 42 checks.

Definition 45 (Moded Head). *Given a head* H *for procedure* p *with type declaration* $p(T_1, \ldots, T_n)$*, the* ***moded head*** H' *is constructed as follows:*

1. ***Assign modes:*** *For each argument position* i*, assign mode* ↓ *if* T_i *is an input type (written with trailing* `?`*) and mode* ↑ *if* T_i *is an output type. Modes propagate through nested term structure according to the type definition, complementing at embedded* `?` *annotations.*
2. ***Complement variables:*** *Replace each variable with its paired variable.*

Example 46 (Moded Head) *Consider the first* `merge` *clause head*
`H = merge([X|Xs], Ys, [X?|Zs?])`
with type `merge(Stream?,Stream?,Stream)`*. Arguments 1 and 2 are consumed (*`Stream?`*), argument 3 is produced (*`Stream`*), yielding the I/O-moded term:*
`↓merge(↓[↓X|Xs], Ys, ↑[↑X?|Zs?])`
Complementing each variable yields the moded head:
`H' = ↓merge(↓[↓X?|Xs?], Ys?, ↑[↑X|Zs])`

Without step 2 (variable complementation), the resulting term `↓merge(↓[↓X|Xs], Ys, ↑[↑X?|Zs?])` places a writer `X` at a consumed position and a reader `X?` at a produced position.

Remark 47 (Mode Correspondence) *Let H' be a moded head constructed per Definition 45 from head H and type declaration D. For any term path $x \in paths(H')$ ending at position p, and corresponding type path $y \in paths(D)$ at the same position p, the mode at p in x equals the mode at p in y.*

The same property holds for body unit goals: each body unit goal A is a produced moded term—mode $\uparrow$ at the root, with consumed embedded positions at $\downarrow$—matching the type's expectation that body unit goals are produced; the root mode is not compared (Definition 42).

Consequently, when checking path consistency (Definition 42) for a moded head or body unit goal, the "mode of the corresponding type symbol" can be read directly from the term path.

4.2 Type Automaton

A GLP type definition D (whether for a type T or its dual $T?$) induces a *type automaton*—a deterministic finite automaton that recognises the regular set of moded paths $paths(D)$. States correspond to types (both T and its dual $T?$), procedures (p/n), and primitives. Transitions are labelled with functor, arity, argument position, and mode.

To simplify type checking, we require type definitions to be deterministic: alternatives must be distinguishable by their top-level functor.The formal automaton construction appears in Appendix I.

4.3 Well-Typing

With these notions, we can define when a clause is well-typed.

Definition 48 (Type Assignment). *For a variable V occurring at position p in a clause C with type declaration D, the **type of** V is the state reached in the type automaton for D by the path from the root to p; where that path passes a wildcard state, `_` or `_?`, before reaching p, the type of V is that wildcard.*

Definition 49 (Well-Typed Clause). *Let $C = (H :- B)$ be a GLP clause and D a GLP type for all its procedures. Then C is **well-typed** by D if:*

1. *There is a moded head H' corresponding to H that is well-typed by D.*
2. *For each unit goal $A \in B$, the produced moded term A' corresponding to A is well-typed by D.*
3. *For every variable pair X and $X?$ in C:*
   (a) *If both occur in the head, or both occur in the body, their types are dual.*
   (b) *If one occurs in the head and the other in the body, they have the same type.*

Definition 410 (Input Prefix, Input-Accepting Clause). *The **input prefix** of an input path x is its prefix up to its first $\uparrow$ edge: the prefix ending at the vertex that edge leaves, or x itself where no edge of x is $\uparrow$. A clause C with moded head H' **accepts** an input path $x \in paths(D)$ if H' has a path consistent with the input prefix of x.*

Term matching (Definition 213) examines a goal against a head along the input prefix: at a $\uparrow$ position the goal holds a writer, which takes the head's

subterm whole, so what lies below the first mode inversion is the clause's output, which no input reaches. Rows 1 and 2 of the table compare a path ending at a functor: the head has that functor there, or a reader.

For example, the input type path `(0,↓) → merge → (1,↓) → "."/2 → (1,↓) → _?` (a non-empty first stream) is accepted by the first `merge` clause (whose moded head has term path `(0,↓) → merge → (1,↓) → "."/2 → (1,↓) → X?`, consistent by rows 1 and 2 of the table) and by the third clause (where `X?` at position 1 matches any ↓-mode subtree). It is not accepted by the fourth clause, whose first argument is the empty list `[]` rather than `"."/2`. The input path `(0,↓) → send → (2,↓) → ch → (2,↑) → []` of `send` (Section 6.3) has input prefix `(0,↓) → send → (2,↓) → ch`, accepted by its one clause, whose moded head has `ch` there and produces a list cell below.

Example 411 (Well-typed Clause) *We verify the first* `merge` *clause is well-typed:*

```
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
```

Condition 1 (Head well-typed): *The moded head from the example is:*

```
H' = ↓merge(↓[↓X?|Xs?], Ys?, ↑[↑X|Zs])
```

Each path in H' is consistent with a path in paths(D), as shown above.

Condition 2 (Body unit goals well-typed): *The body unit goal* `merge(Ys?, Xs?, Zs)` *as a produced moded term* `↑merge(Ys?, Xs?, Zs)` *has paths consistent with* `merge(Stream?,Stream?,Stream)`.

Condition 3 (Variable compatibility): *All four variable pairs satisfy the head-head/head-body dispositions of Definition 49 condition 3:* `X`/`X?` *are head-head duals;* `Xs`/`Xs?`, `Ys`/`Ys?`, *and* `Zs`/`Zs?` *are head-body same-type.*

Definition 412 (Well-Typed GLP Program). *A typed GLP program $P = (Cs, D)$ is* ***well-typed*** *if:*

1. *Every clause $C \in Cs$ is well-typed by D.*
2. *Every input path in D is accepted by some clause $C \in Cs$.*

A procedure declared with `Term?` at a position has an input path for every functor there, so in a well-typed program one of its clauses has a variable at that position or above it: the procedure accepts every consumed term.

4.4 Subtyping

Definition 49 requires that body-body variable pairs have dual types. This requirement can be relaxed using *subtyping*: it suffices that the type of the writer X be a subtype of the dual of the type of its paired reader $X?$. Informally, anything the writer `X` produces, the reader `X?` can consume. Where a subterm reverses the direction of communication, the subtyping direction reverses with it. The same relaxation applies to a head-body pair, whose two occurrences carry the same mode: what the head occurrence receives must be within what the body occurrence accepts. It applies also to a head-head pair, which passes an input to an output: what the input receives must be within what the output declares.

A file system monitor handling `read`, `write`, and `delete` operations has type:

```
FileOp ::= read(Path?,Content)
         ; write(Path?,Content?)
         ; delete(Path?).
```

with procedure declaration `procedure fs_monitor(Stream(FileOp)?)` (where `Stream(T)` denotes a parameterised stream of `T` values; see Section 6).A read-only client that only issues `read` operations has the narrower type `ReadOp ::= read(Path?,Content)`, with declaration `procedure ro_client(Stream(ReadOp))`. Connecting them directly violates strict duality (`ReadOp` $\neq$ `FileOp`), yet the interaction is safe: the client produces a subset of what the monitor accepts, and the monitor's responses (the `Content` in `read`) are exactly what the client expects.

Definition 413 (Simple Prefix). *A **simple prefix** of an output type T is a path in T's type automaton starting from T that contains no mode inversions. A simple prefix ends when it reaches either:*
- *The produced primitive `_`, or*
- *A mode inversion point: any position whose type is marked with `?`.*

Definition 414 (Prefix Acceptance). *A simple prefix p of type A is **accepted by** type B if B has a simple prefix q with identical functor/position structure, where endpoints satisfy: `_` matches only `_`; output type S matches S or `_`; a constant matches `String`; mode inversion $S?$ matches any mode inversion at the same position.*

Definition 415 (Subtyping). *Let A and B be GLP output types. We say A is a **subtype of** B, written $A <: B$, if:*
1. *Every simple prefix of A is accepted by B.*
2. *For every mode inversion point in A reached by a simple prefix—say, type $A'?$ at that position—there is a corresponding mode inversion point $B'?$ in B at the same position, and B' is a subtype of A'.*

$<:$ is the greatest relation satisfying conditions 1 and 2.

Subtyping extends to input types by complementation: $A? <: B?$ if $B <: A$.

A type whose paths are produced throughout and end at a constant, at `Integer`, `Real`, `String` or `Module`, or at `Term` is a subtype of `Term`, so a term of such a type is passed where a `Term` is expected.

Decidability follows from the finiteness of the type DFA: $<:$ is checked by a finite simulation between the DFAs of A and B.

Subtyping generalises exact duality: if $S = T$ then $S <: T$. The relation is transitive but not symmetric: `ReadOp` $<:$ `FileOp` but not conversely.

Definition 416 (Well-Typed Clause with Subtyping). *A GLP clause is **well-typed with subtyping** if it satisfies Definition 49 with condition 3 replaced by: for every variable pair X and $X?$ in C,*

(a) if both occur in the head, the dual of the type of X is a subtype of the type of $X?$; if both occur in the body, the type of X is a subtype of the dual of the type of $X?$;

(b) if one occurs in the head and the other in the body, the dual of the type of the head occurrence is a subtype of the dual of the type of the body occurrence.

Definition 417 (Well-Typed GLP Program with Subtyping). *A typed GLP program $P = (Cs, D)$ is **well-typed with subtyping** if it satisfies Definition 412 with "well-typed clause" replaced by "well-typed clause with subtyping" (Definition 416).*

Remark 418 (Subtyping Variance) *The coinductive structure of Definition 415 embodies the standard variance principle: covariant in output positions (the subtype's functors must be contained in the supertype's), contravariant in input positions (at mode inversion points, the containment direction reverses). This matches the variance of session type subtyping [17].*

5 GLP Semantics and Well-Typing

We derive denotational semantic notions for which the type system is an abstraction. The semantic objects we employ—moded terms—are richer than in LP: they record both partial-run behaviour (so well-typing constrains runs that may fail, deadlock, or never terminate) and the produced/consumed distinction (supporting both output conformance and input coverage).

For a procedure declaration $\mathtt{p}(T_1?, \ldots, T_k?, T_{k+1}, \ldots, T_n)$, input paths traverse the consumed arguments $T_1?, \ldots, T_k?$ and output paths traverse the produced arguments $T_{k+1}, \ldots, T_n$.

5.1 Well-Typed Outcomes

The moded-atoms outcome and moded resolvent of a run (Definition 215) are **well-typed** by D if the moded terms G_0' and G', respectively, are well-typed by D. A goal G_0 is **well-typed** by D if it is well-typed as a body. Note that G_0 can be conjunctive, but there is only a single program D. The **well-typed moded-atom semantics** of a typed GLP program $P = (Cs, D)$ is the set of all moded-atoms outcomes of every finite proper run of P with a well-typed initial goal.

5.2 Soundness of Well-Typing

Next we connect syntactic well-typing with the semantic *output conformance* of runs from well-typed initial goals, and prove it sound.

Definition 51 (Conformance). *A moded path x **conforms to** a GLP type D if either*

(i) x is consistent with some path in $paths(D)$ (Definition 42); or

(ii) x ends at a variable, and deleting that variable leaves a prefix agreeing in functor, arity, and non-root mode with a proper prefix of some path in $paths(D)$.

A variable at a position of its own mode—a writer at a produced position, a reader at a consumed one—is consistent by rows 2 and 3 of the table, so case (*ii*) admits the **pending** variables: a reader at a produced position, the consumer's handle on an output the run has not yet produced, and a writer at a consumed position, the producer's handle on an input the environment has not yet supplied.

Both come from a clause head, whose output placeholders and input slots term matching hands to a goal's writer with the head's subterm they sit in. The partner of a pending variable occurs in the resolvent, or in the outcome where the head paired the two, as the head `new_channel(ch(Xs?, Ys), ch(Ys?, Xs))` of Appendix H does. Case (*ii*) does not relax Definition 42, which stays strict and still rejects a reader at a produced position, or a writer at a consumed one, in a moded head—a mode error—because clause well-typing is defined by consistency, not conformance.

The **type** of a variable occurrence in a moded term is the state its position reaches (Definition 48), complemented where the occurrence is pending. In a source clause head every variable is pending, and condition 3 of Definition 49 says that every variable pair of a well-typed clause has dual types in this sense; so does a well-typed goal, in which no variable is pending.

Lemma 52 (Well-Typing Preservation) *Let $P = (Cs, D)$ be well-typed and $\rho : (G_0, \emptyset) \to \cdots \to (G_n, \sigma_n)$ a proper run from a well-typed initial goal G_0. Then for every $i \leq n$:*

1. *every path of the moded-atoms outcome $G_0\sigma_i^\star$ conforms to D (Definition 51);*
2. *the moded resolvent $G_i\sigma_i^\star$ is well-typed by D as a body; and*
3. *every variable pair with an occurrence in $(G_0$ `:-` $G_i)\sigma_i^\star$ has dual types: the writer's occurrences have one type, and the reader's its dual.*

Proof. By induction on i, with one observation on the terms $\sigma^\star$ moves, stated first.

Transfer. Let T be a term whose paths conform to D below a position of state S, and let $\sigma^\star$ place T at a position of state S or of state $S?$. Every path through T conforms at the new position: the modes of a position are those of its state (Definition 215), so the functors and constants of T, consistent with the paths of S, are consistent with those of $S?$, which differ in modes only; a variable of T keeps its standing, pending or not, at state S and flips it at state $S?$, where its position carries the complemented mode; and each variable keeps its type, the complement of its state exactly where it is pending.

Base ($i = 0$): $\sigma_0 = \emptyset$, so outcome and resolvent are both G_0'. As a well-typed initial goal, G_0' is well-typed by D as a body: every path of each unit goal is consistent, giving (1) and (2), and its variable pairs have dual types with no occurrence pending, giving (3).

Communicate leaves σ unchanged, hence the outcome, so (1) carries over. A reader $X? \in G_{i-1}$ is replaced by T, with $\{X? := T\} \in \sigma_{i-1}$ the counterpart of an assignment $X := T$ an earlier Reduce made. By (2) $X?$ sits at a consumed position, of some state $U?$, and by (3) X has type U; the Reduce bound X either as a goal writer at a produced position of state U, to the head's subterm or output placeholder T there, or as a head input slot of state $U?$, to the subterm or reader T a goal supplied there. In the first case T conforms below state U by condition 1 of Definition 49 and is placed at $U?$, where Transfer makes its variables conventional and its paths consistent; in the second T conforms below $U?$ by (2) and is placed at $U?$, keeping its standing. Either way (2) holds and the types of (3) are unchanged.

Reduce. Unit goal $A \in G_{i-1}$ reduces with the first applicable clause $C = (H \text{ :- } B)$, renamed apart, via writer mgu $\hat{\sigma}$; $G_i = (G_{i-1} \setminus \{A\} \cup B)\hat{\sigma}$ and $\sigma_i = \sigma_{i-1} \circ \hat{\sigma}?$. Since P is well-typed, C satisfies the three conditions of Definition 49; by (2) the writers of A sit at produced positions and its readers at consumed ones.

Matching A against H (Definition 213) walks the two term trees jointly from each argument, the joint positions carrying equal modes in A' and H' (Remark 47), and stops at a variable. A writer of A, at a produced position, is bound to the head's subterm or output placeholder there, and nothing below it is examined; a reader of A, at a consumed position, is bound into the head's input slot there, a term of H there having suspended the match; a term of A meets a term of H of the same functor, the walk continuing, or an input slot, bound to it, and at a produced position meets an output placeholder only by failing. So $\hat{\sigma}$ binds writers of A and the input slots of H the walk reaches, and no variable of B, which the renaming keeps apart from G_{i-1}.

(2). G_i is $G_{i-1} \setminus \{A\}$, unchanged, together with B: by condition 2 each goal of B is a produced moded term well-typed by D, and by condition 3(a) its body–body pairs have dual types. Hence $G_i\sigma_i^\star$ is well-typed as a body.

(1) and (3). The outcome changes only through the pair completion of $\hat{\sigma}?$. A writer X_1 of A, at a produced position of state S and so of type S, is bound to a head subterm T_2 at a position of that state, conforming below it by condition 1, or to an output placeholder $X_2?$ there, of type $S?$ in H and paired by condition 3(b) with a writer X_2 of type S in B. An input slot X_2 of H, at a consumed position of state $U?$ and so of type U, is bound to a subterm or reader T_1 of A at a position of that state, conforming below it by (2); its reader $X_2?$ is in B, of type $U?$ by condition 3(b), or in H at a produced position of state U, of type $U?$ by condition 3(a), inside a subterm handed to a writer of A. By (3) every occurrence of a bound variable and of its partner in $(G_0 \text{ :- } G_i)\sigma_i^\star$ lies at a position of its state or of the dual state, so Transfer places the bound term at each of them with every path through it conforming and every variable keeping its type; $X_2?$ placed for $X_1?$ takes the standing $X_1?$ had, and so the type $S?$ it has in H. The pairs of C enter with the types the clause assigns them, dual by condition 3. Hence every refined path of the outcome conforms, and every pair has dual types, giving (1) and (3).

Theorem 53 (Soundness of Well-Typing) *If a typed GLP program $P = (Cs, D)$ is well-typed, then every output path in its well-typed moded-atom semantics conforms to D (Definition 51).*

This is the covariant, or subsumption, direction. Well-typing soundly over-approximates the semantics: it guarantees type-conformant output, not freedom from failure or deadlock.

Proof. By Lemma 52(1), every path of the moded-atoms outcome $G_0\sigma_i^\star$ of a run from a well-typed initial goal conforms to D, in particular every output path; these outcomes constitute the well-typed moded-atom semantics.

Soundness has a contravariant counterpart for inputs.

Proposition 54 (Input Coverage) *Let $P = (Cs, D)$ be well-typed. Then every input path in $paths(D)$ is accepted by some clause in Cs, and every input path of every clause's moded head is consistent with some path in $paths(D)$.*

Soundness contains a program's outputs within D; coverage requires every input D declares, up to its first mode inversion, to be matched by some clause head. This mirrors session subtyping, covariant in outputs and contravariant in inputs [17]. Soundness is a property of the moded-atom semantics, coverage of the clause heads. Coverage holds per input path, not per argument tuple, so a goal matching no clause may still fail.

Proof. The first conjunct is condition 2 of Definition 412: a clause accepts an input path $x \in paths(D)$ when its moded head has a path consistent with the input prefix of x (Definition 410). The second is condition 1 of Definition 49: each moded head is well-typed by D, so each of its paths, in particular each input path, is consistent with some path in $paths(D)$. Equality of non-root modes (Definition 42) makes each consistent path an input path.

Corollary 55 (Extension to Subtyping) *Theorem 53 holds with "well-typed" replaced by "well-typed with subtyping" (Definition 417).*

Proof. Definition 416 says, in the terms of Lemma 52(3), that in every variable pair of a clause the writer's type is a subtype of the dual of the reader's: for a body–body pair and a head–head pair by its clause (a), both occurrences of a head–head pair being pending, and for a head–body pair by its clause (b), the head occurrence being pending. The lemma holds with (3) so weakened: Transfer then places a term conforming below a type S' at a position of state S or $S?$ with $S' <: S$, and conformance carries over by Definition 415—on produced structure the subtype's functors are contained in the supertype's (condition 1), and at each mode inversion the containment reverses (condition 2), which is the reversal the dual position makes (Remark 418). The theorem follows as before.

6 Parameterised Types

Monomorphic `Stream` (carrying elements of type `_`) cannot express that `merge` preserves the element type from inputs to output; parameterised types recover this precision. Here, parameterised types are syntactic sugar: each use with concrete type arguments expands to a fresh monomorphic type definition before type automaton construction, after which existing well-typing and subtyping machinery apply unchanged. This is *monomorphisation*, the standard implementation of parametric type-polymorphism [25], used by C++, Rust, Go and Futhark, and by MLton for Standard ML: MLton applies all functors and eliminates polymorphism by code duplication, compiling the result in a simply-typed, first-order intermediate language [9,53]. GLP is polymorphic in the sense ML is. The source language quantifies over types; the language actually checked does not.

6.1 Parameterised Type Definitions

A parameterised type definition introduces type parameters, written as uppercase identifiers in parentheses after the type name:

```
Stream(X) ::= [] ; [X | Stream(X)].
```

This defines a type *template*. An *instantiation* supplies a concrete type for each parameter:

```
Stream(Integer)     % stream of integers
Stream(AgentMsg)    % stream of agent messages
```

Multiple parameters are supported:

```
Pair(A, B) ::= pair(A, B).
Channel(In, Out) ::= ch(In, Out?).
```

Mode annotations within a template (`Out?`) are preserved during expansion.

Definition 61 (Parameterised Type Definition). *A **parameterised type definition** has the form $T(X_1, \ldots, X_k) ::= A_1; \ldots; A_n$, where the **type parameters** $X_1, \ldots, X_k$ may occur in place of type names within $A_1, \ldots, A_n$.*

Where a parameterised type refers to itself, directly or transitively, no parameter may occur as a proper subterm of an argument of the self-referential occurrence. Each type instantiation then expands to finitely many type definitions.

This restriction, and the monomorphic-recursion rule its procedure-level counterpart imposes (Section 6.3), are one restriction stated twice, and they are the frontier of the technique rather than a peculiarity of GLP: monomorphisation is possible exactly up to polymorphic recursion, which [25] detects as cyclic flow in the propagation of type instantiations.

6.2 Expansion

Expansion rule. Given a parameterised type definition $T(X_1, \ldots, X_k) ::= A_1; \ldots; A_n$ and an instantiation $T(S_1, \ldots, S_k)$, the expansion is:

$$T\langle S_1, \ldots, S_k\rangle ::= A_1[S_1/X_1, \ldots, S_k/X_k] \ ; \ \ldots \ ; \ A_n[S_1/X_1, \ldots, S_k/X_k]$$

where $T\langle S_1, \ldots, S_k\rangle$ is a fresh type name and $A_i[S_j/X_j]$ denotes simultaneous substitution. Every occurrence of $T(X_1, \ldots, X_k)$ in the body is replaced by $T\langle S_1, \ldots, S_k\rangle$.

Example: `Stream(Integer).` From the template `Stream(X) ::= [] ; [X | Stream(X)].`, the instantiation `Stream(Integer)` expands to:

```
Stream<Integer> ::= [] ; [Integer | Stream<Integer>].
```

Its type automaton is constructed by the rules of Section 4.2, and well-typing proceeds as in Section 4. As expansion preserves the regular-set-of-moded-paths structure of types, the soundness of well-typing (Theorem 53) extends to parameterised types.

6.3 Parameterised Procedure Declarations

A procedure declaration may use type parameters to express uniform behaviour. It names them in a list after the keyword:

```
procedure(X) merge(Stream(X)?, Stream(X)?, Stream(X)).
```

The named parameters are universally quantified, and each may occur bare or within a template instantiation anywhere in the argument types. The same list follows `exported` and `imported`. A declaration with no parameters is written without a list, which is the common case, and a template instantiation whose arguments are all concrete needs none.

Definition 62 (Instantiation). *Let A be a goal in a monomorphic clause C, calling a procedure q whose declaration has parameters $X_1, \ldots, X_k$. A map θ from those parameters to types of the program is an **instantiation** of A if C and the clauses of q are well-typed (Definition 49; with subtyping, Definition 416) when q's declaration is replaced by its expansion under θ, and every input path of that declaration is accepted by some clause of q (Definition 410).*

Two types with the same automaton bind a parameter consistently whatever their names or defining modules, type identity being structural. The bindings the sites of a call supply conflict exactly when no θ makes the clause well-typed with subtyping; so a stream of a subtype merges into a stream of its supertype through `merge(Stream(X)?, Stream(X)?, Stream(X))`, with `X` bound to the supertype. The declaration is a *clause template*. A call in a clause body to a parameterised procedure induces an instantiation of the callee. The *instantiations* of a program form the least set containing an instantiation of every call in a monomorphic clause and closed under this induction, except that recursion is monomorphic: a call to a procedure already being instantiated on the current cycle is checked at that instantiation rather than inducing a new one, and a call that would require a different instantiation of such a procedure is a type error. Recursion thus induces no instantiation, and the non-recursive calls induce finitely many, so the set is finite. For each instantiation in the set, the clauses of the instantiated procedure must be well-typed (Definition 49) by the monomorphic declaration the instantiation produces; Section 6.4 casts this as the well-typing of programs. Every procedure instantiation a run reaches lies in the set, so soundness (Theorem 53) extends to parameterised procedures.

Declaration parameters. The parameters of a procedure declaration are exactly those its parameter list names. An undefined type name occurring in a declaration and not in its parameter list is an error, so a misspelt type name is rejected rather than read as a parameter.

Naming them is what makes that rejection possible. Inferring them instead—taking the parameters to be the undefined names occurring within the arguments of template instantiations—cannot distinguish a parameter from a typo, because a declaration carries no other evidence of which was meant: `=(X, X?)` and `pass(Strem?, Strem)` are the same declaration up to renaming, and no rule reading either tells them apart. With the list, they differ in what they declare: the first names `X` and is legal, the second names nothing and `Strem` is an

undefined type. Inference also fails on a parameter that occurs only bare: `X` in `=(X, X?)` lies within no template instantiation, so an inferred reading makes the declaration parameterless and its type name unresolved, while `X` there is doing the work of tying the two arguments to one type, which the wildcard cannot do. Type definitions already name their parameters, so procedure declarations agree with them rather than adding a further idea.

6.4 Programs and Modules

Parameterised type definitions and parameterised procedure declarations are syntactic sugar; they have no well-typing. Well-typing (Definition 412) is defined only after expansion (Section 6.2), on a typed GLP program (Cs, D) (Definition 31) in which no free type parameter remains: every type reached is a concrete user-defined or primitive type. The unit of type checking is the program: a checker expands it—collecting the instantiation set above and replacing every parameterised reference by its expanded monomorphic name—and applies Definition 412 to the resulting (Cs, D). A fragment that still contains a free type parameter after expansion is not a program and is not type-checked. Checking a parameterised procedure with its parameter left free—equivalently, treating the parameter as the wildcard `_`—is unsound, because a clause that inspects the parameter is then accepted vacuously. A parameterised procedure acquires concrete types, and is checked, only within a program that instantiates it; with every type then concrete, the variable-pair conditions of Definition 49 apply, so a writer/reader mode mismatch in an instantiated clause is rejected.

6.5 Modular Checking via Opaque Types

The per-instantiation rule above checks a parameterised procedure once for each concrete type its program supplies, so a procedure with no caller in its program goes unchecked. A procedure that never inspects its parameters can instead be checked once, for all instantiations at once. Treating a parameter as the wildcard `_` to this end is unsound (Section 6.4); treating it as an *opaque type* is sound, and enforces the parametricity discipline: the procedure passes a parameter-typed value through, and matches no functor or constant against it.

Definition 63 (Opaque Type). *An **opaque type** is a type-automaton state with no outgoing transitions, distinct from the wildcard* `_`*. As the endpoint of a simple prefix it matches only itself and* `Opaque` *(Definition 414), and not the wildcard. The built-in type* `Opaque` *is the union of all opaque types: a state with no outgoing transitions, distinct from the wildcard and from every opaque type, matched by itself and by every opaque type, and by nothing else; as the endpoint of a simple prefix it matches only itself.* `Opaque` *may stand at any position of a type. For a type T, T° is T with every opaque type replaced by* `Opaque`*, and the subtyping relation of Definition 415 is extended by $T <: T^\circ$ for every type T.*

By Definition 42, a moded term path is consistent with an opaque type at a position exactly when the term there is a variable of the position's mode: a variable is consistent with any same-mode type symbol (rows 2 and 3), whereas a functor or constant has no transition to take and matches no wildcard.

Definition 64 (Parametrically Well-Typed). *The **opaque instance** of a parameterised procedure declaration replaces each type parameter by a distinct opaque type. A parameterised procedure is **parametrically well-typed** if, by its opaque instance, its clauses are well-typed with subtyping (Definition 416) and every input path is accepted by some clause (Definition 410).*

Lemma 65 (Parametricity) *Let a parameterised procedure be parametrically well-typed, with no type parameter occurring as an alternative of a type definition. Then for every instantiation—by opaque types, where a term of a type S may stand at a position of type S°—by the monomorphic declaration the instantiation produces, the clauses of the instantiated procedure are well-typed with subtyping (Definition 416) and every input path is accepted by some clause (Definition 410).*

Proof. Let σ map each opaque parameter to its type at the instantiation. Take a clause well-typed with subtyping by the opaque instance and a moded-head or body-goal term path of it. At a position typed by a concrete type, σ leaves the type unchanged, so consistency (Definition 42) is preserved. At a position typed by an opaque parameter the term is a variable of the position's mode; σ replaces the parameter by a type of the same mode, with which a variable is consistent (rows 2 and 3), so consistency is preserved. Each head–head pair dual by the opaque instance stays dual under σ, which acts equally on a parameter and its complement. For the other pairs it suffices that $S <: T$ by the opaque instance implies $S\sigma <: T\sigma$, σ commuting with complementation. Let R hold of $(S\sigma, T\sigma)$ for every such $S <: T$, and of (U, U) for every type U; R satisfies the two conditions of Definition 415, and so is contained in $<:$, the greatest relation satisfying them. Condition 1: a simple prefix of $S\sigma$ either is a simple prefix of S whose endpoint σ leaves a concrete type or a mode inversion point, and is accepted by $T\sigma$ as it is by T; or it passes an output opaque type α of S and continues within $\alpha\sigma$, and then the prefix of S ending at α is accepted by T, so T has at the same position either α, an opaque type matching only itself and `Opaque`, and $T\sigma$ continues there with the same automaton $\alpha\sigma$, which accepts the rest, or `Opaque`, which $\alpha\sigma$, an opaque type by the hypothesis, matches. Where $S <: T$ holds by $T = S^\circ$, $T\sigma = S^\circ = (S\sigma)^\circ$, since σ replaces each opaque parameter by an opaque type and S has no other opaque type, so $S\sigma <: T\sigma$ by the same clause. Condition 2: a mode inversion point of $S\sigma$ reached by a simple prefix either is the image of one of S, of type $A'?$, where T has $B'?$ with $B' <: A'$, so that $T\sigma$ has $B'\sigma?$ and $(B'\sigma, A'\sigma)$ is in R; or it lies within some $\alpha\sigma$, where $T\sigma$ has the same inversion point at the same type U, and (U, U) is in R. A type definition's determinism—distinct top-level functors—can change under σ only where a parameter is an alternative, excluded by hypothesis, so every type definition stays deterministic. Input-path acceptance transfers by the same step: a clause accepting an opaque-instance input path carries a variable at each opaque-parameter position on it, and a variable accepts every refinement of that path under σ, and so of its input prefix (Definition 410). Hence each instantiated clause is well-typed with subtyping and every input path of the instantiation is accepted.

A parametrically well-typed procedure is thus checked once and certified for every instantiation; the linked program (Section 6.4) need not re-check it, and soundness follows by Corollary 55. A procedure that inspects a parameter—placing a functor or constant at a parameter position—is not parametrically well-typed, and is checked per instantiation. A parameter occurring as an alternative of a type definition is likewise checked per instantiation, its determinism resting on the instantiation. The opaque-parameter route is therefore the sole means of certifying a parametric procedure outside a program: a parameter-inspecting procedure has no well-typing, acquiring one only per instantiation, so a checker presented with it and no instantiation has nothing to certify. Where a program contains a parameterised procedure that no call in it instantiates and that is not parametrically well-typed, compilation rejects the program.

6.6 Subtyping

Since parameterised types expand to monomorphic types, subtyping is handled by the DFA-based subtype check (Section 4.4). **Example: subtyping through expansion.** Suppose:

```
Msg ::= text(String) ; cmd(String).
Cmd ::= cmd(String).
```

Then `Cmd` <: `Msg` by the standard subtype check. Expanding `Stream(Cmd)` and `Stream(Msg)` yields:

```
Stream<Cmd> ::= [] ; [Cmd | Stream<Cmd>].
Stream<Msg> ::= [] ; [Msg | Stream<Msg>].
```

The DFA-based subtype check determines that `Stream<Cmd>` <: `Stream<Msg>`, because at each list-cons step, the head type `Cmd` is a subtype of `Msg`. This is the standard covariance for output positions, with no variance annotation.

Contravariance from modes. For input positions, the mode annotation `?` reverses the subtype direction through complementation. Consider:

```
Channel(In, Out) ::= ch(In, Out?).
```

A `Channel(Msg, Cmd)` expands to `ch(Msg, Cmd?)`. A `Channel(Msg, Msg)` expands to `ch(Msg, Msg?)`. Since `Cmd` <: `Msg`, by complementation `Msg?` <: `Cmd?`. Therefore `Channel(Msg, Msg)` <: `Channel(Msg, Cmd)`. This is the standard contravariance, obtained by complementation.

7 Conclusion

Our *moded-atom semantics* extends the standard Herbrand atom semantics with modes and with meaning for partial computations, and our *moded types* are regular sets of moded paths over it. Our central result (Theorem 53) establishes the soundness of well-typing—every output path in a well-typed program's well-typed moded-atom semantics conforms to its declared type—and Lemma 52 shows well-typing is preserved through GLP computation. Proposition 54 is the contravariant counterpart: every declared input path is matched by some clause head.

We extended the core type system with two mechanisms for programming in the large. The *module system* (Section F) supports separate type checking of modules against locally declared interfaces, with structural type compatibility ensuring that independently developed modules can be composed safely. Compilation flattens a module hierarchy into a single compiled module by renaming; dynamic activation admits a module arriving at run time as a value, on equality of the hashes of declaration type automata (Section F.3). *Parameterised types* (Section 6) enable generic definitions—`merge` over `Stream(X)` for any `X`—with variance determining how subtyping propagates through type parameters.

When implementing Typed GLP by AI, gaps found at the code level drove revisions to the specification and the definitions. When using Typed GLP by AI, the main benefit was eliminating a class of mode errors that previously manifested only as silent runtime suspensions.

Future Work. The canonical serialisation of type automata, their hashes, a module's export tables and the `run` and `find_type` primitives are implemented (Appendix J); type checking their call sites is future work. Activating a guest whose types its host does not hold needs no dynamic type checking under the invitation discipline, its types being opaque to the host (Section F.3); activation outside that discipline would need it, and is left to future work. The interaction between parameterised types and subtyping across module boundaries deserves further study, particularly for higher-order cases where type parameters themselves carry mode structure.

Data Availability Statement.

The type checker, the module system and the typed programs of this paper are in the public GLP repository, whose link is withheld for anonymity; an artifact will be submitted for evaluation.

References

1. Aiken, A., Lakshman, T.K.: Directional type checking of logic programs. In: Proceedings of the 1994 International Symposium on Static Analysis (SAS). Lecture Notes in Computer Science, vol. 864, pp. 43–60. Springer (1994)
2. Baker, H.G., Hewitt, C.: The incremental garbage collection of processes. In: Proceedings of the 1977 Symposium on Artificial Intelligence and Programming Languages. pp. 55–59. ACM (1977). https://doi.org/10.1145/800228.806932
3. Balzer, S., Pfenning, F.: Manifest sharing with session types. Proceedings of the ACM on Programming Languages **1**(ICFP), 37:1–37:29 (2017). https://doi.org/10.1145/3110281
4. Blinn, A., Li, X., Kim, J.H., Omar, C.: Statically contextualizing large language models with typed holes. In: Proceedings of the ACM on Programming Languages (OOPSLA). vol. 8, pp. 1–29. ACM (2024). https://doi.org/10.1145/3689746
5. Brady, E.: Type-Driven Development with Idris. Manning Publications (2017)
6. Bruynooghe, M., Janssens, G.: An instance of abstract interpretation integrating type and mode inferencing. In: Proceedings of the 5th International Conference and Symposium on Logic Programming. pp. 669–683. MIT Press (1988)
7. Caires, L., Pfenning, F.: Session types as intuitionistic linear propositions. In: Proceedings of the 21st International Conference on Concurrency Theory (CONCUR). Lecture Notes in Computer Science, vol. 6269, pp. 222–236. Springer (2010). https://doi.org/10.1007/978-3-642-15375-4_16

8. Cardelli, L., Wegner, P.: On understanding types, data abstraction, and polymorphism. Computing Surveys **17**(4), 471–523 (1985). https://doi.org/10.1145/6041.6042
9. Cejtin, H., Jagannathan, S., Weeks, S.: Flow-directed closure conversion for typed languages. In: Smolka, G. (ed.) Programming Languages and Systems (ESOP 2000). Lecture Notes in Computer Science, vol. 1782, pp. 56–71. Springer, Berlin, Heidelberg (2000). https://doi.org/10.1007/3-540-46425-5_4
10. Chen, Z., Pfenning, F.: Colf logic programming as infinitary proof exploration. arXiv preprint arXiv:2510.12302 (2025)
11. Clark, K., Gregory, S.: Parlog: parallel programming in logic. ACM Transactions on Programming Languages and Systems (TOPLAS) **8**(1), 1–49 (1986). https://doi.org/10.1145/5001.5390
12. Debray, S.K.: QD-Janus: A sequential implementation of Janus in Prolog. Software: Practice and Experience **23**(12), 1337–1360 (1993)
13. Fowler, M.: Understanding spec-driven-development: Kiro, spec-kit, and tessl. MartinFowler.com (October 2025), https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html, analyzes the shift toward using formal specifications and types as the primary interface for AI code generation
14. Friedman, D.P., Wise, D.S.: The impact of applicative programming on multiprocessing. Indiana University Computer Science Department Technical Report (TR-26) (1976)
15. Frühwirth, T., Shapiro, E., Vardi, M.Y., Yardeni, E.: Logic programs as types for logic programs. In: Proceedings of the 6th Annual IEEE Symposium on Logic in Computer Science (LICS). pp. 300–309. IEEE Computer Society (1991). https://doi.org/10.1109/LICS.1991.151654
16. Gaifman, H., Shapiro, E.: Fully abstract compositional semantics for logic programs. In: Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages. pp. 134–142 (1989)
17. Gay, S., Hole, M.: Subtyping for session types in the pi calculus. Acta Informatica **42**(2–3), 191–225 (2005). https://doi.org/10.1007/s00236-005-0177-z
18. Girard, J.Y.: Linear logic. Theoretical Computer Science **50**(1), 1–101 (1987). https://doi.org/10.1016/0304-3975(87)90045-4
19. Google: Dart programming language. https://dart.dev (2024)
20. Honda, K.: Types for dyadic interaction. In: Proceedings of the 4th International Conference on Concurrency Theory (CONCUR). Lecture Notes in Computer Science, vol. 715, pp. 509–523. Springer (1993)
21. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. Journal of the ACM **63**(1), 9:1–9:67 (2016). https://doi.org/10.1145/2827695
22. Knuth, D.E.: Literate programming. The Computer Journal **27**(2), 97–111 (1984). https://doi.org/10.1093/comjnl/27.2.97
23. Lafont, Y.: Interaction nets. In: Conference Record of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL). pp. 95–108. ACM (1990)
24. Levi, G., Palamidessi, C.: The semantics of the read-only variable. In: Proc. Symposium on Logic Programming. pp. 128–137. IEEE (1985)
25. Lutze, M., Schuster, P., Brachthäuser, J.I.: The simple essence of monomorphization. Proceedings of the ACM on Programming Languages **9**(OOPSLA1), 1015–1041 (Apr 2025). https://doi.org/10.1145/3720472
26. Meyer, B.: Object-Oriented Software Construction. Prentice Hall, 2nd edn. (1997), introduces Design by Contract

27. Mierowsky, C., Taylor, S., Shapiro, E., Levy, J., Safra, M.: On the implementation of flat concurrent prolog. Proceedings of the 1985 Symposium on Logic Programming pp. 276–286 (1985)
28. Milner, R.: Communicating and Mobile Systems: The π-Calculus. Cambridge University Press (1999)
29. Mishra, P.: Towards a theory of types in prolog. In: Proceedings of the 1984 International Symposium on Logic Programming. pp. 289–298. IEEE (1984)
30. Mündler, N., Guerraoui, R., Vechev, M.: Type-constrained code generation with language models. In: Proceedings of the 46th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). ACM (2025), available at https://arxiv.org/abs/2504.09246
31. Mycroft, A., O'Keefe, R.A.: A polymorphic type system for Prolog. Artificial Intelligence **23**(3), 295–307 (1984). https://doi.org/10.1016/0004-3702(84)90017-1
32. Pierce, B.C.: Types and Programming Languages. MIT Press (2002)
33. Plotkin, G.D.: A structural approach to operational semantics. Aarhus university (1981)
34. Reddy, U.S.: A typed foundation for directional logic programming. In: Schroeder-Heister, P. (ed.) Extensions of Logic Programming. Lecture Notes in Computer Science, vol. 475, pp. 282–318. Springer (1992)
35. Saraswat, V.A., Kahn, K.M., Levy, J.: Janus: A step towards distributed constraint programming. In: Proceedings of the 1990 North American Conference on Logic Programming (NACLP). pp. 431–446. MIT Press (1990)
36. Saraswat, V.A., Rinard, M., Panangaden, P.: Semantic foundations of concurrent constraint programming. In: Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL). pp. 333–352. ACM (1991). https://doi.org/10.1145/99583.99627
37. Shapiro: Volition elicitation: Operational semantics for people and their machines (2026), https://arxiv.org/abs/2607.14138
38. Shapiro, E.: A subset of concurrent prolog and its interpreter. ICOT Technical Report, TR-003 (1983)
39. Shapiro, E.: The family of concurrent logic programming languages. ACM Computing Surveys (CSUR) **21**(3), 413–510 (1989). https://doi.org/10.1145/72551.72555
40. Shapiro, E.: Grassroots distributed systems: Concept, examples, implementation and applications (brief announcement). In: 37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391). pp. 47:1, 47:7. LIPICS, Italy (2023)
41. Shapiro, E.: Glp: A grassroots, multiagent, concurrent, logic programming language for ai (full version). arXiv preprint arXiv:2510.15747, Summary to appear in Proc. of ICLP'26 (2025)
42. Shapiro, E.: Grassroots platforms with atomic transactions: Social graphs, cryptocurrencies, and democratic federations. In: Proceedings of the 27th International Conference on Distributed Computing and Networking. pp. 71–81 (2026). https://doi.org/10.1145/3772290.3772309, arXiv preprint arXiv:2502.11299
43. Shapiro, E.: Implementing grassroots logic programs with multiagent transition systems and ai (full version). arXiv:2602.06934. Summary to appear in Proc. of LOPSTR+PPDP'26 (2026)
44. Silverman, W., Hirsch, M., Houri, A., Shapiro, E.: The logix system user manual version 1.21. In: Concurrent Prolog: Collected Papers, pp. 46–77 (1988)

45. Somogyi, Z., Henderson, F., Conway, T.: The execution algorithm of mercury: An efficient purely declarative logic programming language. Journal of Logic Programming **29**(1–3), 17–64 (1996). https://doi.org/10.1016/S0743-1066(96)00068-4
46. Tick, E.: The deevolution of concurrent logic programming languages. The Journal of Logic Programming **23**(2), 89–123 (1995)
47. Ueda, K.: Guarded horn clauses. In: Logic Programming '85. Lecture Notes in Computer Science, vol. 221, pp. 168–179. Springer (1986). https://doi.org/10.1007/3-540-16479-0_17
48. Ueda, K.: Resource-passing concurrent programming. Proceedings of TACS 2001 pp. 95–126 (2001). https://doi.org/10.1007/3-540-45500-0_5
49. Ueda, K., Morita, M.: Moded flat ghc and its message-oriented implementation technique. New Generation Computing **12**(4), 337–368 (1994). https://doi.org/10.1007/BF03038307
50. Ueda, K., Morita, M.: I/o mode analysis in concurrent logic programming. In: Proceedings of the International Symposium on Theory and Practice of Parallel Programming. pp. 356–368. Springer (1995). https://doi.org/10.1007/BFb0026579
51. Wadler, P.: Propositions as sessions. In: Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (ICFP). pp. 273–286. ACM (2012). https://doi.org/10.1145/2364527.2364568
52. Wadler, P.: Propositions as sessions. Journal of Functional Programming **24**(2–3), 384–418 (2014). https://doi.org/10.1017/S095679681400001X
53. Weeks, S.: Whole-program compilation in MLton. In: Proceedings of the 2006 Workshop on ML (ML '06). p. 1. Association for Computing Machinery, Portland, Oregon, USA (2006). https://doi.org/10.1145/1159876.1159877, invited talk
54. YARDENI, E., FRUEHWIRTH, T., SHAPIRO, E.: Polymorphically typed logic programs. In: Logic Programming: Proceedings of the Eighth International Conference. pp. 379–393. MIT Press (1991)
55. Yardeni, E., Shapiro, E.: A type system for logic programs. Journal of Logic Programming **10**(2), 125–153 (1991). https://doi.org/10.1016/0743-1066(91)90013-G

A Logic Programs

We recall standard Logic Programs (LP) notions of syntax and semantics via goal reduction.

A.1 Syntax

We employ the standard LP notions of variables, constants, terms, clauses, procedures, and programs.

Definition A1 (Logic Programs Syntax). *Let $\mathcal{V}$ denote the set of **variables** (identifiers beginning with uppercase). A **term** is a variable, a constant (numbers, strings, or the empty list* `[]`*), or a compound term $f(T_1, \ldots, T_n)$ with functor f and subterms T_i. Let $\mathcal{T}$ denote the set of all terms. We use standard list notation:* `[X|Xs]` *for a list cell,* `[X1,...,Xn]` *for finite lists. A term is **ground** if it contains no variables.*

*A **unit goal** is a compound term or a string, also called an **atom**. A **goal** is a multiset of unit goals; the empty goal is written* `true`*. A **clause** A `:-` B has head A (a unit goal) and body B (a goal); a **unit clause** has empty body. A **logic program** is a finite set of clauses; clauses for the same predicate form a **procedure**. Let $\mathcal{G}(P)$ denote the set of goals over program P.*

Example A2 (Append) *The standard logic program for list concatenation is the following procedure:*

```
append([X|Xs], Ys, [X|Zs]) :- append(Xs, Ys, Zs).
append([], Ys, Ys).
```

Logically, a clause `A :- B` *is a universally-quantified implication in which* `B` *implies* `A`*, and a program is a conjunction of its clauses. By convention, we use plural variable names like* `Xs` *to denote a list of* `X`*'s.*

A.2 Operational Semantics

A *substitution* σ is an idempotent function $\sigma : \mathcal{V} \to \mathcal{T}$. By convention, $\sigma(x) = x\sigma$. Let Σ denote the set of all substitutions. We assume the standard notions of instance, ground, renaming, renaming apart, unifier, and most-general unifier (mgu). We assume a fixed renaming-apart function, so that the result of renaming T' apart from T is well defined.

Definition A3 (LP Goal/Clause Reduction). *Given an LP unit goal* A *and clause* C*, with* H `:-` B *being the result of renaming* C *apart from* A*, the* ***LP reduction*** *of* A *with* C ***succeeds with*** (B, σ) *if* A *and* H *have an mgu* σ*.*

We define the operational semantics of LP and of GLP via transition systems; in both, set relations and operations refer to multisets.

Definition A4 (Transition System [41]). *A* ***transition system*** *is a tuple* $TS = (C, c_0, T)$ *where:*
- C *is an arbitrary set of* ***configurations***
- $c_0 \in C$ *is a designated* ***initial configuration***
- $T \subseteq C \times C$ *is a* ***transition relation****. A transition* $(c, c') \in T$ *is also written as* $c \to c' \in T$*.*

A transition $c \to c' \in T$ *is* ***enabled*** *from configuration* c*. A configuration* c *is* ***terminal*** *if no transitions are enabled from* c*. A* ***computation*** *is a (finite or infinite) sequence of configurations where for each two consecutive configurations* (c, c') *in the sequence,* $c \to c' \in T$*. We write* $c \xrightarrow{*} c'$ *to denote the existence of a computation (empty if* $c = c'$*) from* c *to* c'*. A* ***run*** *is a computation starting from* c_0*, which is* ***complete*** *if it is infinite or ends in a terminal configuration. The* ***outcome*** *of a complete run is determined by a domain-specific function from complete runs to an outcome space.*

Definition A5 (Logic Programs Transition System). *A transition system* $LP(P) = (C, c_0, T)$ *is a* ***Logic Programs transition system*** *for a logic program* P *and initial goal* $G_0 \in \mathcal{G}(P)$*, if* $C = \mathcal{G}(P) \times \Sigma$*,* $c_0 = (G_0, \emptyset)$*, and* T *is the set of all transitions* $(G, \sigma) \to (G', \sigma') \in (\mathcal{G}(P) \times \Sigma)^2$ *such that for some unit goal* $A \in G$ *and clause* $C \in P$ *the LP reduction of* A *with* C *succeeds with* $(B, \hat{\sigma})$*,* $G' = (G \setminus \{A\} \cup B)\hat{\sigma}$*, and* $\sigma' = \sigma \circ \hat{\sigma}$*.*

The following notion of a proper run ensures that variable names are not re-used.

Definition A6 (Proper and Successful Run, Outcome [41]). *A run* $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ *of* $LP(P)$ *is* ***proper*** *if for any* $1 \leq i < n$*, a variable that occurs in* G_{i+1} *but not in* G_i *also does not occur in any* G_j*,* $j < i$*. If proper, the* ***outcome*** *of* ρ *is* $(G_0$ `:-` $G_n)\sigma_n$*. Such a run is* ***successful*** *if* $G_n = \emptyset$*.*

The following proposition justifies calling a proper LP run a *derivation*, and a complete proper run ending in the empty goal a *successful derivation*.

Proposition A7 (LP Computation is Deduction) *The outcome* $(G_0$ `:-` $G_n)\sigma$ *of a proper run* $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ *of* $LP(P)$ *is a logical consequence of* P*.*

Note that the LP transition system does not require the initial goal G_0 to be a unit goal.

A.3 Denotational Semantics

The $LP(P)$ transition system allows defining several denotational semantic notions for a program P:

1. The *clause semantics* of P is the set of all outcomes of all proper runs of $LP(P)$ with an initial most-general unit goal (arguments are distinct variables). It is closely related to the fully-abstract compositional semantics of LP [16].
2. The *atom semantics* of P is the set of all outcomes of all successful derivations of $LP(P)$ with an initial most-general unit goal.
3. The *ground atom semantics* of P is the standard model-theoretic semantics of logic programs. It is the set of ground instances of the atom semantics of P over the Herbrand universe of P.

A.4 Types

We associate with each logic term T a *term tree*: a labelled tree with vertices labelled with the functor/arity for compound terms and constants/variables for leaves in T, and edges labelled with the argument index of the subterm.

The *paths* of a term T, $paths(T)$, is the set of labelled paths in its term tree. For example, in `append([X|Xs],Ys,[X|Zs])`, the first argument contributes the path `append --1--> "."/2 --1--> X`.

In [15], a type for LP is defined as a regular set of paths, and a program is well-typed if the type includes all paths of all atoms in its ground-atom semantics. Types are specified using well-founded BNF rules and procedure type declarations, for example:

```
List ::= [] ; [_|List].

procedure append(List,List,List).
```

with `_` denoting any path in any ground term in the Herbrand universe of the program. Note that LP well-typing is *covariant*: it guarantees that all outputs are within the type, but does not constrain inputs. Thus `append([],c,c)` is in the ground-atom semantics, and `append` is well-typed by `append(_,_,_)`. So, providing useful type declarations is up to the programmer or program designer:

the more informative and tight the type declaration, the more useful is the statement that the program is well-typed. Many extensions for this notion of LP types are possible and needed, in particular polymorphic types, discussed in [54].

B Grassroots Logic Programs: Further Material

This appendix carries the material of Section 2 that the body omits.

Monotonicity. Key differences between LP and cGLP relate to monotonicity. In LP, if a goal cannot be reduced, it will never be reduced. In cGLP, a goal that cannot be reduced now may be reduced in the future: if A and H have an mgu that writes on a reader $X? \in A$, and therefore have no writer mgu at present, another goal that has X may reduce, assigning X, and later $X?$, to a value that will allow A and H to have a writer mgu. Conversely, in LP, if a goal A can be reduced now with some clause $H :- B$, with a regular mgu of A and H, it may not be reducible in the future due to variables that A shares with other goals being assigned values by other goal reductions, preventing unification between the instantiated A and H. In cGLP, if a goal A can be reduced now (with a writers mgu), it can always be reduced in the future, as the SO invariant ensures that no other goal can assign any writer in A.

In an implementation, if a cGLP goal A cannot be reduced now, but there is a readers substitution σ such that $A\sigma$ can be reduced, such readers are identified, the goal A *suspends* on these readers, and is rescheduled for another reduction attempt once any of them is assigned.

Despite these differences, cGLP has the same notion of logical consequence as LP. Let /? be an operator that replaces every reader by its paired writer.

Proposition B1 (GLP Computation is Deduction) *Let $(G_0 \text{ :- } G_n)\sigma_n^\star$ be the outcome of a proper run $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ of cGLP(P). Then $(G_0 \text{ :- } G_n)\sigma_n^\star/?$ is a logical consequence of $P/?$.*

B.1 Guards

GLP clauses may include *guards*—tests that determine clause applicability.

Definition B2 (Guarded Clause). *A **guarded clause** has the form H :- G | B, where H is the head, G is a conjunction of guard predicates, and B is the body. The guard separator "|" distinguishes guards from the body, and is interpreted logically as a conjunction. Guard arguments are readers paired to head writers.*

Guards have three-valued semantics. Each guard predicate defines its *success* condition. A guard *suspends* if it does not succeed but some instance of it under a readers substitution would succeed. A guard *fails* if no such instance exists. A guard conjunction succeeds if all members succeed; it suspends if any member suspends and none fail; it fails if any member fails.

Definition 28 of a GLP goal/clause reduction is augmented to succeed if the guard also succeeds.

Remark B3 (Guards and SRSW) *Guard occurrences count toward SRSW satisfaction: if X? occurs in a guard, its paired writer X must occur in the head and X? may additionally occur once in the body.*

Furthermore, if the success of a guard implies that X? is bound to a ground term, then both X and X? may occur multiple times in the clause. Groundness-implying guards include `ground`*,* `integer`*,* `number`*,* `string`*,* `constant`*, arithmetic comparisons (*`<`*,* `>`*,* `=<`*,* `>=`*,* `=:=`*,* `=\=`*), and ground equality (*`=?=`*). Guards such as* `known` *and* `compound` *do not imply groundness:* `known(X?)` *succeeds when X? is bound to* `f(Y?)` *with Y? unbound.*

Guard predicate type signatures appear in the GLP language paper [41].

C Typed GLP: Further Material

This appendix carries the material of Section 3 that the body omits.

C.1 Parameterised Types in Brief, and Declaration Placement

To ease the specification of types, we include parameterised types, a limited form of type polymorphism. A parameterised type declaration of `merge` could look like:

```
Stream(X) ::= [] ; [X|Stream(X)].

procedure(X) merge(Stream(X)?,Stream(X)?,Stream(X)).
```

Declaration placement. Type declarations may appear anywhere in a program before they are used. A procedure declaration must appear immediately before its defining clauses, which must be contiguous. For example:

```
Stream ::= [] ; [_|Stream].

procedure merge(Stream?,Stream?,Stream).
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge(Xs, [], Xs?).
merge([], Ys, Ys?).

procedure copy(Stream?, Stream).
copy([X|Xs], [X?|Ys?]) :- copy(Xs?, Ys).
copy([], []).
```

C.2 SRSW Relaxations in Typed GLP

Anonymous variables. An *anonymous variable* is any variable whose name begins with `_` (e.g., `_`, `_In`, `_Out`). Anonymous variables may appear anywhere a writer variable may appear. Each occurrence denotes a fresh writer with no paired reader, the one exception to the SRSW restriction: an input a clause does not read is dropped. Anonymous readers (`_?`, `_Name?`) are not permitted in a clause, in the head or the body: a reader whose writer is never written is a goal that can never be satisfied, so a clause writes every writer it holds, closing a stream it does not extend [41]. Values assigned to anonymous variables are discarded. For example:

```
second([_, X | _], X?).
foo(X) :- bar(_Result, X?).
```

In the first clause, `_` discards the head and tail of the input list. In the second, `_Result` discards the first output of `bar`.

The symbols `_` and `_?` in type definitions are primitive type symbols, not variables. **Readers of constant types, a future extension.** A *constant type* is a type each of whose alternatives is a constant, one of the primitive types `Integer`, `Real`, `String` and `Module`, or a constant type. No alternative of a constant type carries a functor, so every term assigned to a variable of such a type is a constant, and a constant contains no variable. GLP will admit several occurrences of a reader of constant type in a clause, its paired writer occurring once: the writer is assigned once, what it is assigned is a constant, so no occurrence of the reader produces a second occurrence of a writer, and SO preservation (Proposition 211) will hold by its own argument. It is the relaxation Remark B3 states from the guard's side: there the groundness of the assigned term follows from the success of a guard, here it will follow from the type of the variable. The language does not have this extension now: a reader of constant type occurs once, as any reader does, unless a guard implies it ground.

A reader of type `MutualRef` may occur more than once: the writer a mutual reference holds is the runtime's, no program can reach it, and every write through it is a kernel's, so no occurrence of the reader gives a program a second writer.

Type checking of guards. For the purpose of type checking, guards are treated as conjunction with the body: `H :- G | B` is type-checked as `H :- G, B`. A guard atom that tests the type of a head occurrence narrows it. Let S be the type of the occurrence and T the type declared for the position it occupies in the guard. The guard atom is well-typed if the **meet** of S and T—the type whose paths are the paths of both, again a type, since alternatives are distinguished by their top-level functor—is non-empty, and the occurrence has that meet as its type in the body, where condition 3 of Definition 49 is applied to it. Otherwise a guard that discriminates a union is unwritable: `module(M?)` at an argument of type `Content ::= String ; Module` tests the case its clause is for, and asking `Content` <: `Module` refuses it.

Type checking of `:=`. The root declares the body kernel `:=(Number, Exp?)` (Appendix H), which assigns to its writer the value of an arithmetic expression. A goal `X := E` is checked with the type of `X` taken as `Integer` where `E` is an *integer expression*—an integer literal, a reader of type `Integer`, one of `+`, `-`, `*`, `//`, `mod` or `abs` over integer expressions, or `integer`, `round`, `floor` or `ceil` over any expression—, as `Real` where `E` is a *real expression*—a real literal, a reader of type `Real`, one of `/`, `real`, `sqrt`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `exp`, `ln` or `log` over any expression, one of `+`, `-`, `*` or `pow` with an operand a real expression, or `abs` over a real expression—and as `Number` otherwise; so `N1 := N? + 1` with `N` an `Integer` types `N1` as `Integer`, which the monitor of Section C.3 needs, and `Z := X? + Y?` with `X` and `Y` of type `Real` types `Z` as `Real`. `pow` over integers is a `Number`, a negative exponent yielding a real. The rule is sound because the kernels evaluating these operators yield an integer on integers and a real where

an operand is a real, `integer`, `round`, `floor` and `ceil` yield an integer on any number, and `/`, `real` and the functions named yield a real on any number, by the catalogue [41]; it is a rule for one kernel, as the narrowing by a guard is, since the result type of `:=` depends on the functors of its expression, which no declaration expresses: a procedure has one declaration, and a union admits each functor once.

Type checking of `=..` and `..=`. The root declares `=..(_, Stream(_)?)`, which composes a term from the list of its functor and its arguments, and `..=(Stream(_), _?)`, which decomposes a term into that list [41]; a term so composed or decomposed may have variables of both modes, so the two are declared with wildcards. A goal `T =.. L` is checked at `=..(Term, Stream(Term)?)` if `L` is a *list of terms*—the empty list, a reader of a type that is a subtype of `Stream(Term)`, or a list cell whose head is a term expression and whose tail is a list of terms—and a goal `L ..= T` is checked at `..=(Stream(Term), Term?)` if `T` is a *term expression*—a constant, a reader of a type that is a subtype of `Term`, or a structure or a list cell over term expressions; so `T` .. [F?|As?]= with `F` a `String` and `As` a `Stream(Term)` types `T` as `Term`. The rule is sound because a term composed of a functor and of produced terms is a produced term, and the functor and the arguments of a consumed term are consumed terms; it is a rule for these two system predicates, as the rule above is for `:=`, since the type of the term composed depends on the types of the elements of the list, which no declaration expresses.

C.3 Typed GLP Programming Examples

Stream merging. Combining the elements above provides a typed GLP `merge` program:

```
Stream ::= [] ; [_|Stream].

procedure merge(Stream?,Stream?,Stream).
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge(Xs, [], Xs?).
merge([], Ys, Ys?).
```

This `merge` program is well-typed (Section 4). It is also well-typed with weaker type declarations such as `procedure merge(_?,_?,_)`, but is not well-typed with declarations that violate mode requirements, such as `procedure merge(_,_,_)`.

So, as in LP, providing useful type declarations is up to the programmer.

Monitors with hollow messages. Next, we show an interactive type with mode complementation. Consider a monitor process, maintaining a local state and serving requests to update the state and read it. It can serve arbitrarily many clients through a network of merge processes. Monitor queries use a programming technique called a *hollow message*—a message that includes a writer, with which the consumer responds to the producer. Note that the program relaxes the SRSW requirement for consumed integers.

```
CounterCall::= add ; clear ; read(Integer?).
```

```
procedure monitor(Stream(CounterCall)?).
monitor(In) :- monitor(0,In?).

procedure monitor(Integer?,Stream(CounterCall)?).
monitor(N,[add|In]) :- N1 := N? +1, monitor(N1?,In?).
monitor(_,[clear|In]) :- monitor(0,In?).
monitor(N,[read(N?)|In]) :- integer(N?) | monitor(N?,In?).
```

Further examples are in the repository, under `programs/book/`: a bounded buffer by hollow messages (`streams/buffered_communication/hollow_integers.glp`), cooperative stream construction with interactive types (`streams/producers_consumers/coop_stream.glp` and `cooperative.glp`), difference lists (`recursive/list_processing/quicksort.glp`), and bidirectional channels (`streams/producers_consumers/relay.glp` and `streams/objects_monitors/network_switch_3way.glp`).

D GLP Well-Typing: Further Material

This appendix carries the examples of Section 4 that the body omits.

Example D1 (Moded Clauses for `merge`) *We illustrate moded clauses for the* `merge` *program. Each moded clause consists of a moded head H' (constructed per Definition 45) and moded body unit goals (each a produced moded term).*

Clause 1 (recursive): `merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).`
Moded clause:
`↓merge(↓[↓X?|Xs?], Ys?, ↑[↑X|Zs]) :- ↑merge(Ys?, Xs?, Zs).`
Variable types: `X:_`, `X?:_?` *(head-head, duals);* `Xs`,`Xs?:Stream` *(head-body, same);* `Ys`,`Ys?:Stream?` *(head-body, same);* `Zs`,`Zs?:Stream` *(head-body, same).*

Clause 3 (base case): `merge(Xs, [], Xs?).`
Moded clause:
`↓merge(Xs?, ↓[], ↑Xs).`
Variable types: `Xs:Stream`, `Xs?:Stream?` *(head-head, duals).*

In each moded clause, the head is I/O-moded (arguments 1 and 2 consumed, argument 3 produced), and variables are replaced by their paired forms to match position modes. All variable pairs satisfy type consistency: head-head pairs have dual types, head-body pairs have the same type.

Example D2 (Subtyping Analysis) *We verify that* `ReadOp` *is a subtype of* `FileOp`. `ReadOp` *has functor* `read/2` *only, while* `FileOp` *includes* `read`, `write`, *and* `delete`.

Condition 1: *Each simple prefix of* `ReadOp` *is accepted by* `FileOp`*: position 1 reaches* `Path?` *(mode inversion) in both types; position 2 reaches* `Content` *(output) in both.*

Condition 2: *At the mode inversion point (*`Path?`*), both types have identical* `Path`*, so the contravariant check holds trivially.*

Thus `ReadOp` $<:$ `FileOp`*, and a writer of type* `ReadOp` *paired with a reader of type* `FileOp?` *is well-typed.*

E Parameterised Types: Further Material

This appendix carries the material of Section 6 that the body omits.

Example: `Channel(FriendMsg, FriendMsg).` From `Channel(In, Out) ::= ch(In, Out?)`, the instantiation `Channel(FriendMsg, FriendMsg)` expands to:
`Channel<FriendMsg,FriendMsg> ::= ch(FriendMsg, FriendMsg?).`

The channel's second component is an input stream from the holder's perspective.

Nested parameterised types. Parameters may themselves be parameterised type instantiations:
`Stream(Pair(Integer, String))`

Expansion proceeds inside-out: first expand `Pair(Integer, String)` to `Pair<Integer,String> ::= pair(Integer, String).`, then expand `Stream(Pair<Integer,String>)` to:

```
Stream<Pair<Integer,String>> ::= [] ; [Pair<Integer,String>
                                     | Stream<Pair<Integer,String>>].
```

Example: instantiation from call context. Consider a typed program with:

```
AgentMsg ::= befriend(Constant, Response?)
           ; connected(Constant)
           ; rejected.

procedure agent_merge(Stream(AgentMsg)?,
                      Stream(AgentMsg)?,
                      Stream(AgentMsg)).
agent_merge(A, B, C) :- merge(A?, B?, C).
```

The call `merge(A?, B?, C)` occurs where `A`, `B`, `C` have type `Stream(AgentMsg)`. The type checker matches `Stream(X)` against `Stream(AgentMsg)` and infers `X = AgentMsg`. It then expands the declaration to:
`procedure merge(Stream<AgentMsg>?, Stream<AgentMsg>?, Stream<AgentMsg>).`
and type-checks the clause body against this monomorphic declaration.

Channel operations. The root `self.glp` declares channel operations using the two-parameter `Channel(In, Out)` template:

```
procedure(X, Y) send(X?, Channel(Y, Stream(X))?,
                     Channel(Y, Stream(X))).
procedure(X, Y) receive(X, Channel(OpenStream(X), Y)?,
                        Channel(Stream(X), Y)).
procedure(X, Y) new_channel(Channel(X, Y), Channel(Y, X)).
```

In `send`, `X` is the message element type and `Y` the read stream type; the type checker infers both from the call context. `receive` consumes a message from a non-empty read stream (`OpenStream(X)`); the defined guard `close` covers the closed one (Appendix H).

The clause `new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).` produces two channel endpoints: what one end writes, the other reads, and vice versa.

Multiple parameters.
`procedure(X) relay(Stream(X)?, Stream(X), Channel(X, X)?).`

If the context provides `Stream(Integer)?` for the first argument and `Channel(String, String)?` for the third, the type checker reports an error: `X` cannot be both `Integer` and `String`.

E.1 Programs and Linked Programs

Definition E1 (Program, Linked Program). *A **program** is a directory carrying a* `self.glp`*, or a self-contained module (Appendix F.1), with one or more concrete initial goals. Its **linked program** is the typed GLP program (Definition 31) obtained by parameterised-type expansion (Section 6.2), an instantiation of every call to a parameterised procedure (Definition 62), and the linking step of compilation (Appendix F.2), defined only when expansion leaves no free type parameter.*

Definition E2 (Well-Typed Program). *A program is **well-typed** if its linked program is well-typed (Definition 412).*

Soundness (Theorem 53) is a property of the linked program, hence of the program, and makes no claim about a module checked outside a program.

E.2 Interaction with Modules

Imported declarations instantiate parameters. A module that imports `merge` instantiates its type parameter to the specific message type used locally:

```
CounterCall ::= inc ; dec ; read(Integer).

imported procedure streams#merge(Stream(CounterCall)?,
                                 Stream(CounterCall)?,
                                 Stream(CounterCall)).
```

The `streams` module need not know about `CounterCall`. Its parameterised declaration `merge(Stream(X)?, Stream(X)?, Stream(X))` is instantiated to `Stream(CounterCall)` at the importing site.

Structural compatibility across modules. Because type identity is structural (Section F.1), independently expanded types are compatible if their type automata are equal, subtyping applying to the arguments of a call (Section F.1). If module A expands `Stream(Msg)` and module B independently defines `MsgStream ::= [] ; [Msg  MsgStream].|`, the two types have identical automata and are compatible.

Example: monitor with parameterised merge. Consider a monitor that accepts requests via a merged input stream:

```
%% counter_monitor.glp
CounterCall ::= inc ; dec ; read(Integer).

imported procedure streams#merge(Stream(CounterCall)?,
                                 Stream(CounterCall)?,
                                 Stream(CounterCall)).

exported procedure counter(Stream(CounterCall)?).
counter(In) :- ...
```

A client module imports the monitor and provides its own typed stream:

```
%% client.glp
CounterCall ::= inc ; dec ; read(Integer).

imported procedure counter_monitor#counter(Stream(CounterCall)?).

procedure client(Stream(CounterCall)).
client([inc, inc, read(R?) | Rest?]) :- ...
```

Both modules independently expand `Stream(CounterCall)` to the same monomorphic type.

E.3 Expansion Algorithm

The expansion algorithm runs as a preprocessing step, after parsing and before type automaton construction.

1. **Collect templates.** Scan all type definitions. A definition whose name is followed by a parenthesised parameter list is a template; all others are monomorphic. Templates are recorded but not added to the type environment.
2. **Collect instantiations.** Scan all type definitions, procedure declarations (including `imported` and `exported` declarations), and type definition bodies for parameterised type references $T(S_1, \ldots, S_k)$. Each distinct instantiation is recorded. A type definition body may reference a different parameterised type (e.g., a `Pair` appearing inside `Stream`); such cross-references are collected in this step and expanded in the next. Scan also every clause body for calls to a parameterised procedure; each call, with the enclosing clause's parameters bound, records an instantiation of the callee (Definition 62), except that a call to a procedure already being instantiated on the current cycle reproduces that instantiation rather than recording a new one. An instantiated procedure's body may call further parameterised procedures, so repeat the clause-body scan over each new procedure instantiation until none is added; recursion records no instantiation, so this terminates.
3. **Expand.** For each recorded instantiation, generate a fresh monomorphic type definition by substituting parameters and renaming. Nested instantiations are expanded inside-out.
4. **Replace references.** In all type definitions and procedure declarations, replace every parameterised type reference $T(S_1, \ldots, S_k)$ with its expanded name $T\langle S_1, \ldots, S_k\rangle$.
5. **Remove templates.** Remove parameterised type definitions from the type environment.

F Module System

This section extends Typed GLP with a module system for type-safe composition of separately developed modules.

Definition F1 (Typed Procedure, Module). *A **typed procedure** is a procedure declaration* `procedure` $p(T_1, \ldots, T_n)$. *immediately followed by a procedure for p/n (Definition 23). A **module** is a sequence of type definitions and typed procedures, in which every type is defined before its use in a declaration, with a subset of the typed procedures **exported**.*

F.1 Design

The module system rests on four principles.

Hierarchy mirrors the file system. A GLP program is organised as a source-code directory hierarchy, with a `self.glp` module in its top directory. Each `.glp` file is a module; each directory is a scope. A distinguished file `self.glp` in any directory defines types and procedures shared by all modules in that directory and its descendants, following FCP's `self.cp` convention [44]. A directory's `self.glp` is the directory's interface: alongside the shared definitions it declares which of the directory's procedures are **exported**, and a call into the directory from outside it resolves only to a procedure its `self.glp` exports. A Typed GLP module is **self-contained** if it does not include a cross-module call `M#p` or an uninstantiated type parameter; self-containment concerns a module's calls and its types, not its scope, so a self-contained module still sees every scope above it. A Typed GLP program is a directory carrying a `self.glp` module, or a self-contained module, with one or more concrete initial goals (Definition E1). Compilation (Section F.2) takes a program and yields a compiled module.

Implicit ancestor scoping. A module sees every type and procedure definition from every ancestor scope without any import directive. The chain is anchored at the root, which lies at or above the directory of the program being compiled, so a program below the root sees the scopes between them.

Self-contained type checking. Every module declares the full moded type of every cross-module procedure it calls, via `imported procedure` declarations. A module may be checked separately against its own text and its ancestor scope; soundness is established on the linked program (Definition E1), the unit of type checking.

Structural type compatibility. Two types with the same type automaton are compatible, regardless of their names or defining modules.

Procedure declarations. Every procedure declaration is one of three kinds:

`procedure p(...).`	private to the module
`exported procedure p(...).`	callable from other modules
`imported procedure M#p(...).`	declares dependency on module M

Types referenced in an `imported` declaration are resolved against the caller's type scope, or, written `M#T`, against the exports of module `M`: `M#T` is the type `T` that a declaration `M` exports references, under `M`'s definition of it, so a module importing a compiled procedure names its types once, by the module that defines them, and writes no copy of them. `M#Procedure` is the type of a goal of a procedure `M` exports held as a term: the union of the declarations `M` exports, the type `Procedure` of Section 3.1 restricted to them, with each argument type complemented, as an end of a channel is held at the dual of its other end's type. A meta-program is parameterised by the type of the goals it interprets and called at it: a meta-interpreter declared `procedure(G) reduce(Goal(G)?)`, with `Goal(G) ::= true ; (Goal(G), Goal(G)) ; G`, is called as `reduce(Goal(M#Procedure)?)` for the program `M` it interprets, and, `G` being an alternative of `Goal(G)`, it is checked at each program it interprets (Section 6.5), the functors of `M`'s

declarations distinct from `true` and `,`. Its clauses read the interpreted program's clauses from a table typed at `G`, `procedure(G) clause(Goal(G)?, Goal(G))`, with the entries `clause(true, true)` and `clause((A, B), (A?, B?))` beside the program's own, since an `otherwise` guard narrows no type. Export and import are declared per procedure; there are no separate export or import lists. A declaration carries the transitive closure of the types its signature references, so types are not exported separately.

External access. A procedure of a program is externally accessible only if the program's `self.glp` exports it. The program's `self.glp` exports a procedure by declaring it `exported` and either defining it directly or giving a forwarding clause whose body is the cross-module call $M\#p$ to a module that defines p, each argument threaded at its declared polarity. Depth does not decide accessibility: a procedure is an entry point exactly when the program's `self.glp` exports it, by definition or by forwarding.

Scope construction. A device nominates one directory as its **root**, and every compilation on that device is under that root. A program lies at or below the root, and the scope of each of its modules runs from the root down to that module.

Definition F2 (Root, Scope). *Let $\mathcal{D}$ be the directories of a file system, $d \preceq d'$ that d is d' or an ancestor of it, and $\mathrm{dir}(M)$ the directory holding a module M. Write d.`self` for the module defined by the `self.glp` of d, undefined where d holds no such file; Π for the GLP language primitives (Appendix H); and $E \sqcup E'$ for the merge of two scopes in which E' shadows E.*

*A **root** is a directory $r \in \mathcal{D}$. For a module M with $r \preceq \mathrm{dir}(M)$, let $d_1, \ldots, d_k$ be the directories from $d_1 = r$ to $d_k = \mathrm{dir}(M)$ in order. The **scope of** M **under** r is*

$$\Sigma_r(M) \;=\; \Pi \sqcup d_1.\texttt{self} \sqcup \cdots \sqcup d_k.\texttt{self} \sqcup M,$$

omitting each d_i.`self` that is undefined.

The root is a property of the device and not of the program. For $r \prec r' \preceq \mathrm{dir}(M)$ the scopes $\Sigma_r(M)$ and $\Sigma_{r'}(M)$ differ in general, by d.`self` for $r \preceq d \prec r'$, and nothing in the file system tells the two apart: a directory without a `self.glp` is no boundary, Σ_r omitting it and continuing past it, and a directory with one is not thereby a root. The root is therefore nominated rather than derived, and what a party receiving a compiled module relies on is the key of the device that compiled it, which that party accepts or rejects [41].

The language primitives and a directory `self.glp` enter a scope by different routes. The **language primitives** are the primitive types, the guard predicates and the body kernels, which the runtime implements (Appendix H.1); they are the base of every scope, for every module of every program, wherever that program sits in a file system. They are not an ancestor directory and are not reached by the path from r. A **directory** `self.glp` is an ordinary module, reached only by that path, and contributes to M's scope exactly when its directory is r or lies between r and M. The root `self.glp` is one, and defines in GLP the predefined types and the kernel and seam predicates [41].

Two consequences follow, and they are independent of each other. The root `self.glp` is in the scope of every module compiled on the device, wherever that module's program sits below the root: it is d_1. And a directory `self.glp` off the path from r to M contributes nothing to M—not its definitions and not the modules it exposes—however near it lies in the file system. A repository whose top `self.glp` both declares the language primitives and holds definitions and `-expose` directives of its own therefore is reached by the two routes: a module compiled under a different root keeps the primitives and loses everything else in it.

Common protocol types are defined once at the lowest common ancestor of all modules that use them.

The `-expose` directive. A `self.glp` may contain the directive `-expose(M).`, which lifts the exported procedures of module `M` (and the types their signatures carry) into that directory's scope, as if defined in its `self.glp`. `M` is a module path (e.g., `lib#streams`), resolved relative to the directory of the `self.glp` containing the directive, within that directory's subtree. Shadowing applies as usual: a procedure the `self.glp` defines shadows one of the same name and arity it exposes; two exposed modules contributing the same name and arity at one level is an error. An exposed procedure is callable by name inside the directory and its descendants, but it is not thereby exported by the program's `self.glp`, so it is an entry point only if the program's `self.glp` exports it in its own right.

Cross-module type checking. A cross-module call `M # p`$(a_1, \ldots, a_n)$ in module N is well-typed if N contains a declaration `imported procedure M#p`$(T_1, \ldots, T_n)$ and for each argument a_i, the type of a_i in the clause context is compatible with T_i under the standard well-typing rules of Section 4, including subtyping with mode variance. This is identical to the well-typing rule for local calls, substituting the imported declaration for a local one. The imported declaration must agree with the declaration `M` exports: their type automata are equal (structural type compatibility) once the exported declaration's type parameters, if any, are instantiated (Definition 62); subtyping applies to the arguments of a call, not between the two declarations. Where the imported declaration names type parameters, as an exported one may (Section 6.3), the call instantiates them as a local call does (Definition 62): a parameter the importing module holds open stays open across the module boundary and is fixed at the call, the clauses of the called procedure being those of the linked program (Definition E1). The qualifier `M` is a single child directory or module file relative to the caller's directory: a directory is entered through its `self.glp`, a module file through its own `exported` declarations. Multi-segment path qualifiers `d1#...#dk#p` are future work.

Example. Consider a social graph application with an agent module and an application module. Shared types are defined in `self.glp`:

```
%% self.glp
Response ::= accept(FriendChannel) ; no.
AgentContent ::= befriend(Constant, Response?)
               ; connected(Constant)
```

```
                ; rejected.
AgentChannel ::= ch(AgentToUserStream, UserToAgentStream?).
```

The agent module exports a single procedure:

```
%% agent.glp
exported procedure agent(Constant?, UserInStream?,
                         NetInStream?, OutputsList?).
procedure(X) merge(Stream(X)?, Stream(X)?, Stream(X)).
```

The application module declares its dependency:

```
%% app.glp
imported procedure agent#agent(Constant?, UserInStream?,
                               NetInStream?, OutputsList?).
```

and calls `agent # agent(alice, UserIn?, NetIn?, Outs)`. The type checker verifies this call against the local `imported` declaration using the types from `self.glp` (visible to both modules via ancestor scoping), without reading `agent.glp`. Both modules may define a private `merge/3` without conflict, since each is scoped to its own module.

F.2 Compilation

Compilation takes a program (Section F.1) and yields a **compiled module**, a value of the primitive type `Module`, or rejects the program and yields none. Linking is a step of it rather than a process after it: a compiled module is a single flat program in which every cross-module call has become a local call.

The transformation proceeds in five steps. First, the compiler collects every `.glp` file of the program's directory tree, together with the `self.glp` of each directory from the root down to the program, and every module a `self.glp` among them exposes, with the `self.glp` of each directory from the root down to that module, and constructs the scope of each module (Definition F2); an exposed module outside the program's tree is compiled in its own scope, as every module is, and a call it makes to a procedure of a `self.glp` on its own chain resolves there. Second, each module is checked against its scope—for SRSW, its guards and its types—exactly as a single-module program is, whether or not an entry point reaches it. Third, after type checking, every procedure p/n and every type T in every `.glp` file (including `self.glp` files) is renamed to $M{:}p/n$ and $M{:}T$, where M is the module's path from the root, eliminating name collisions—for example, both `agent.glp` and `app.glp` may define `merge/3` and a type `Msg`, yielding `agent:merge/3` against `app:merge/3` and `agent:Msg` against `app:Msg`, while `secure/app.glp` yields the distinct `secure/app:merge/3`. Types are renamed on the same argument as procedures: two sibling modules have distinct scopes, so two same-named types defined in them are two types, which one flat namespace would otherwise make one, checking a module against a definition it cannot see. Fourth, every goal in every clause body is resolved: a local call to p in module M becomes $M{:}p$, a cross-module call $M'\#p$, whose qualifier is a single child directory or module file relative to the caller's directory, resolves to the procedure p that the qualifier exports—a directory exports through its `self.glp`, a module file through its own `exported` declarations—renamed to its

prefix (M':p), and a call matching a procedure in an ancestor `self.glp` is resolved to its renamed form by walking the scope; every type reference is resolved the same way, to the renamed type of the nearest scope defining it. A type is named by its name and its number of parameters, as a procedure is named by its name and arity. Exposed procedures resolve like procedures of the exposing directory's `self.glp`. Fifth, the compiler retains the **reachable** procedures: the exported procedures of the program's `self.glp`, and every procedure called in the body of a reachable one. These exports are the compiled module's **entry points**, called by plain name by a goal posted to it. For each entry point the compiler generates an **alias clause**, whose head carries the entry point's plain name and whose body is the single goal naming the renamed procedure, each argument threaded at its declared polarity as in the forwarding clause of Section F.1: a writer in the head and its paired reader in the body at each consumed argument, a reader in the head and its paired writer in the body at each produced one. A single-module program, having no `self.glp`, exports all its procedures, so every one is an entry point. Restricting the program to its reachable procedures is semantically equivalent to the whole.

Sources compiled to GLP. A directory may hold sources in a language compiled to GLP modules, as the `.vglp` sources of [37] are. The types the compilation of a directory's sources forms are defined in that directory's scope, as if its `self.glp` defined them, and so are in the scope of every module at or below that directory, in whatever program it is compiled and when it is loaded alone; a module elsewhere names them by `M#T`, as it names any type of another module's exports. The sources are compiled in the directory's scope without those types, and the directory's `self.glp` is checked after, with them.

The flat program is the linked program of Definition E1, and it is the object checked. Every clause in it satisfies SRSW, as Definition 23 requires of a GLP program, the alias clauses of the fifth step included, and every clause in it is type-checked, an alias clause at the declaration of its entry point. Type checking it verifies each procedure's clauses at the concrete types of every call to it, now that every call is local. This check is more stringent than the per-module check, as it checks the called procedure's clauses against the concrete types the call supplies. A program failing it is rejected, and compilation yields the rejection and no module.

Obtaining a compiled module. Compilation and file reading are language constructs, specified in the GLP language paper [41]. `compile` compiles the program a path names, assigning either the compiled module or the errors that rejected it and, where it succeeds, writing the module as an artefact in that program's own directory; `load_file` reads a file within the calling module's own directory, assigning the module the file holds where it is a certified compiled module, and its text otherwise. A module also reaches an agent as a value over a typed stream, which is the case of Section F.3.

Entry and the absence of a boot module. A directory program is identified by its directory, and that directory is what compilation is given. A compiled module is entered by posting a goal to it, and the goal calls an entry point by

plain name. The module defining the procedure a program starts from is reached like any other: the program's `self.glp` exports that procedure, by definition or by forwarding. No module has a distinguished status as the program's start, so there is no boot module to find, and the procedures that may be posted are exactly the entry points. A module named `boot` is an ordinary module and the name carries no status; several of the applications of Section F.5 have one, and it is reached like any other.

A program whose `self.glp` exports no procedure therefore has no entry points, which the fifth step restricts to the empty set of procedures. No goal resolves against it, so it is not a program in the sense of Definition E1, and compilation rejects it rather than flattening it and reporting success. A directory with no `self.glp` at all is rejected for the prior reason: a program is a directory carrying a `self.glp` or a self-contained module (Section F.1), and such a directory is neither. A directory of modules that is not a program—a library reached by ancestor scoping, or a collection of examples compiled one at a time—is used as those are used, and is not compiled as a program at all.

A single file used to start several agents at once—a test, a demonstration—may carry a clause whose body goals are written $G@p$, naming an initial goal and the agent to run it at, and the persons scripted in GLP that the clause names [43], and nothing else: the agents' program is a program directory, which the agents load. That is invocation notation and not a construct of the language: it says what the runtime starts and where, as a command line does, and the compiler consumes it before type checking. No term of GLP has that form, no module may contain one, and a program directory carries none.

Only a well-typed program (Definition E2) yields a compiled module.

F.3 Dynamic Activation

Dynamic activation runs a goal on a module that is not present at compile time but arrives at run time as a value of the primitive type `Module`. Its type discipline rests on a *type identity*: the hash of the full type automaton of a procedure declaration, in canonical minimised form, with every type the signature references expanded and the procedure-name label abstracted—not of the declaration's text. By structural type compatibility (Section F.1), two declarations are compatible exactly when their automata are equal, hence exactly when their type identities are equal. The name is abstracted because two declarations with the same argument types are compatible whatever they are called: the procedure transition of Definition I2 contributes to the identity its arity, argument position and mode, and not its functor. The two constructs that obtain an identity and perform an activation, `find_type/2` and `run/3`, are language constructs and are specified in the GLP language paper [41], which cites this one for the type identity and the compatibility they rest on. `find_type` assigns the type identity of a declaration in the caller's scope; `run` activates a module value and posts a goal to it where the identity supplied equals the one the module's exports record for the goal's predicate.

The activation check compares hashes; no runtime type checking of terms is introduced. Soundness is conditional: if `Goal` is well-typed by a declaration whose

type identity is `Type`, and `run` activates, then the goal's author and the module hold equal automata for the conversation, and the terms crossing the boundary conform on both sides. The condition is a requirement on the clause that authors the goal: it must have `Goal` well-typed by the declaration of its predicate in scope, with `Type` that declaration's identity, obtained via `find_type`. Under delegation the running module holds no declaration of `Goal`'s predicate: `Goal` and `Type` arrive together over a typed stream, the condition having been enforced where they were authored.

A procedure declared at a scope shared by the goal's author and the module—their lowest common ancestor `self.glp`—has the same automaton on both sides by construction; the hash covers the case with no shared scope, a module shipped from another agent. Within one agent every module was compiled there, and equality of type identities completes the check; for a shipped module, that it is a well-typed module whose exports record true identities rests on the attestation of Section F.4. Section F.4 exchanges this same identity for the types of a conversation; the module source hash exchanged alongside it identifies the code and not the types, so two agents holding one type under two defining sources still match on it.

Opaque types. A program that activates modules it was not compiled with, as a platform hosting applications does, cannot hold a declaration of every type of every module it activates. It need not. Such a **host** and a module it activates, its **guest**, communicate in two ways: with the host, at types the host declares, which the guest uses as they are; and with the executions of the same guest at other agents, at the guest's own types, which are opaque to the host. For the second, the host declares one parameterised procedure, its **entry template**, whose type parameters type the channel between two executions of a guest and every value of the guest's own that passes through the host uninspected, such as a value the guest draws for its person on a channel the host declares; a guest whose values are of several types instantiates a parameter with their union. The host's procedures that handle such values pass them on and never inspect a term of a parameter's type, so they are parametrically well-typed (Definition 64) and checked once for every guest. A guest declares its entry as the template's instance at its own types. A host that holds the ends of several guests in one structure, as a stream of invitations does, types them by `Opaque`, the union of all opaque types (Definition 63), in place of the parameter—`Stream(Opaque)` for ends typed by the parameter itself, `Stream(C(Opaque))` for ends of a channel type `C` over it—and can only pass them on. The type checker takes `Opaque` back to one opaque type in one place: in a goal that `run` activates at an opaque instance at $h_1, \ldots, h_k$, a variable of type T° may stand at a position typed T, for a type T whose opaque types are labelled among $h_1, \ldots, h_k$.

Definition F3 (Type Identity of a Type, Opaque Instance). *The **type identity** $\iota(T)$ of a type T is the hash of its type automaton in canonical minimised form, as for a procedure declaration. Let D be a parameterised procedure declaration with type parameters $X_1, \ldots, X_k$, and $h_1, \ldots, h_k$ type identities. Where no X_i occurs as an alternative of a type definition, the **opaque instance***

$D\langle h_1, \ldots, h_k\rangle$ replaces each X_i by an opaque type (Definition 63) labelled h_i, two such opaque types being one state exactly when their labels are equal; its type identity is that of the resulting declaration, an opaque type printed as its label.

We write $\bar{T}$ for types $T_1, \ldots, T_k$ and $\iota(\bar{T})$ for the list $\iota(T_1), \ldots, \iota(T_k)$, in the order of D's parameters. A guest M records in its exports, for each entry that is a template D's instance at its types $\bar{T}$, the type identity of $D\langle\iota(\bar{T})\rangle$, and with it the list $\iota(\bar{T})$. The host obtains $\iota(\bar{T})$ from the module value and computes the type identity of $D\langle\iota(\bar{T})\rangle$ from its own declaration of D; `run` activates M where the two are equal. The constructs that do so are the GLP language paper's [41]; each takes the list, and refuses one whose length is not the number of D's parameters.

The invitation discipline. An agent running a guest M invites another agent with two things and nothing else: the module identity of M (Section F.4), and one end of a channel of the host's entry template at the guest's types, opaque to the host. The invitee's host activates the same module M and hands it the other end. Both ends run M, so they instantiate the template with the same types, and the attestation of Section F.4 compares the type identity of the template's opaque instance as the type identity of the conversation's root channel. The discipline is what makes the step from `Opaque` sound: the end the invitee's host hands to M arrived with M's module identity, from an execution of M. The step is sound also where the host itself mints a channel at a parameter's type and hands its two ends to two activations of one module value at one list $\iota(\bar{T})$, as a host does for a guest's channel to its person: `opaque_type` yields one list for the two entries, which the host checks, and both ends run M at $\bar{T}$.

Proposition F4 (Opaque Activation) *Let a host's procedures that handle its entry template D's parameters be parametrically well-typed, no type parameter occurring as an alternative of a type definition, and let a guest M be well-typed, its entry declared as D's instance at its types $\bar{T}$. Let* `run` *activate M on a goal the host authors, the type identity of $D\langle\iota(\bar{T})\rangle$ equal to the one M's exports record for its entry, each variable of type S° that the goal places at a position typed S, the opaque types of S labelled among $\iota(\bar{T})$, holding an end received under the invitation discipline with M's module identity, or one end of a channel the host minted whose other end it hands to an activation of the same module value at $\iota(\bar{T})$. Then the host's clauses at $D\langle\iota(\bar{T})\rangle$ and M's clauses are well-typed with subtyping, and every term that crosses between the host and M, or between two executions of M, conforms at $\bar{T}$ to the declaration of the side that reads it, though the host was never checked at $\bar{T}$.*

Proof. By Lemma 65, the host's clauses instantiated at the opaque types labelled $\iota(T_1), \ldots, \iota(T_k)$ are well-typed with subtyping. Equal type identities are equal automata, by structural type compatibility, and an opaque type labelled $\iota(T_i)$ stands for the minimised automaton of T_i, whose hash it is; so the host's declaration of D at those opaque types and M's declaration of its entry have equal automata once each label is read as its type, and a goal the host authors, well-typed by the one, is well-typed by the other. A variable of type S° that

the goal places at a position typed S holds an end that arrived with M's module identity from an execution of M, or one end of a channel the host minted whose other end an activation of the same module value holds, so that both ends run M, where D's parameters type it at $\bar{T}$, and the attestation of Section F.4 compares the root channel's type identity. M's clauses are well-typed by its own declarations, and the host inspects no term of an opaque type. Hence every term that crosses between the host and M, or between two executions of M, conforms at $\bar{T}$ to the declaration of the side that reads it.

The type identity is implemented—automaton expansion, minimisation, canonical serialisation, hash, and the declared and exported tables of a module (Appendix J). The two constructs are implemented and type checking their call sites is future work; compilation ahead of execution (Section F.2) is the model in force.

F.4 Type-Compatible Attestation Between Agents

The guarantee of Theorem 53 requires a well-typed program and a well-typed initial goal. Within a single program both are established at compile time. A term arriving from another agent, however, is produced outside the scope of the receiver's type checker, so for such a term neither hypothesis is fulfilled by the checks above.

Cross-agent communication takes place only between mutually attested runtimes. Participants prove to one another that they run the same module encoding the shared protocol, and that the variables through which they communicate carry the same types, by exchanging the module identity—the hash of its source—and the type identity of the conversation's root channel (Section F.3)—and proceeding only on equality; derived variables undergo no handshake, their types being determined at both ends by the root type and the communications. Equal type identities are equal type automata, hence one type by structural compatibility, so the peer writes every variable of the conversation as a well-typed program under the same type discipline, fulfilling the well-typed-input hypothesis and eschewing runtime typechecking. The architecture, and the theorem that mutually attested well-typed agents compose into a well-typed system, are developed in work in preparation.

The remaining producer of unchecked terms is the initial goal posted to the runtime, at boot or interactively; it is type-checked before execution as a body goal (Definition 49).

F.5 Implementation

The module system has been implemented in Dart as part of the GLP runtime. Ancestor type scoping, the three procedure declaration kinds, and cross-module type checking are operational.

Compilation is implemented as the AST-to-AST transformation of Section F.2.

An earlier dynamic-dispatch mechanism, which routed cross-module calls through channels at runtime, could not be typed without runtime type-checking,

which the language avoids; it was removed from the runtime once dynamic activation (Section F.3) was implemented.

The system is validated by the multi-module programs of the GLP repository's test suite. One is the social graph of `programs/social/graph`, a directory program whose test scenarios—silent plays, befriending, unfriending from either side, and introductions—load through compilation and run.

Before parameterised types, a generic stream utility such as `merge` could not be fully typed: its declaration could not express "this operation preserves the specific type of its input." With the parameterised type declarations of Section 6 it can, and the root `self.glp` declares `procedure(X) merge(Stream(X)?, Stream(X)?, Stream(X)).`

G Deferred Proofs

This appendix contains proofs of propositions from Sections A and 2.

Proposition A7 (LP Computation is Deduction) *The outcome* $(G_0 \texttt{ :- } G_n)\sigma$ *of a proper run* $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ *of* $LP(P)$ *is a logical consequence of* P*.*

Proof. By induction on the length n of the run.

Base case ($n = 0$): The outcome is $(G_0 \texttt{ :- } G_0)\emptyset = (G_0 \texttt{ :- } G_0)$, which is the tautology $G_0 \Leftarrow G_0$.

Inductive step: Suppose the proposition holds for runs of length $n-1$. Consider a run of length n with final transition $(G_{n-1}, \sigma_{n-1}) \to (G_n, \sigma_n)$. By definition, there exist atom $A \in G_{n-1}$ and clause $C = (H \texttt{ :- } B) \in P$ (renamed apart) such that A and H have mgu $\hat{\sigma}$, $G_n = (G_{n-1} \setminus \{A\} \cup B)\hat{\sigma}$, and $\sigma_n = \sigma_{n-1} \circ \hat{\sigma}$.

The clause C, being universally quantified, entails the instance $(H \texttt{ :- } B)\hat{\sigma}$, i.e., $H\hat{\sigma} \Leftarrow B\hat{\sigma}$. Since $A\hat{\sigma} = H\hat{\sigma}$ (by unification), we have $A\hat{\sigma} \Leftarrow B\hat{\sigma}$.

By the inductive hypothesis, the prefix run has outcome $(G_0 \texttt{ :- } G_{n-1})\sigma_{n-1}$, a logical consequence of P. Applying $\hat{\sigma}$ and substituting $A\hat{\sigma}$ with $B\hat{\sigma}$ (justified by the clause instance), we obtain $(G_0 \texttt{ :- } G_n)\sigma_n$, also a logical consequence of P.

Proposition B1 (GLP Computation is Deduction) *Let* $(G_0 \texttt{ :- } G_n)\sigma_n^\star$ *be the outcome of a proper run* $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ *of* $cGLP(P)$*. Then* $(G_0 \texttt{ :- } G_n)\sigma_n^\star/?$ *is a logical consequence of* $P/?$*.*

Proof. The $/?$ operator replaces every reader $X?$ by its paired writer X, transforming GLP terms into LP terms. We show that the cGLP run ρ corresponds to an LP run $\rho/?$ of $LP(P/?)$.

Consider a cGLP transition $(G, \sigma) \to (G', \sigma')$:

- *Reduce transition*: Goal A reduces with clause C via writer mgu $\hat{\sigma}$. Applying $/?$, the clause $C/?$ is an LP clause, and $A/?$ unifies with the head $H/?$ via the mgu $\hat{\sigma}/?$ (since writers map to writers). This is a valid LP reduction.
- *Communicate transition*: A reader $X? \in G$ is replaced by the value T assigned to its paired writer. Under $/?$, both $X?$ and X map to X, so this transition becomes the identity—the variable X is already assigned T in the LP view.

Thus each cGLP transition corresponds to zero or one LP transitions, and the cGLP run ρ projects to an LP run $\rho/?$ of $LP(P/?)$. By Proposition A7, the outcome of $\rho/?$ is a logical consequence of $P/?$.

Proposition 211 (SO Preservation) *If the initial goal G_0 satisfies SO, then every goal in a proper cGLP run satisfies SO.*

Proof. By induction on the length of the run. The base case is immediate: G_0 satisfies SO by assumption.

For the inductive step, assume G satisfies SO and consider a transition $(G, \sigma) \to (G', \sigma')$:

- *Reduce transition*: Goal $A \in G$ reduces with clause $C = (H \texttt{ :- } B)$ via writer mgu $\hat{\sigma}$, yielding $G' = (G \setminus \{A\} \cup B)\hat{\sigma}$. Since C satisfies SRSW, it satisfies SO. Since C is renamed apart from G, the variables in B are fresh. The writer mgu $\hat{\sigma}$ binds writers of A to subterms of H and writers of H to subterms of A, and no other variable; a writer of A occurs nowhere in $G \setminus \{A\}$ (by SO of G) and a writer of H occurs nowhere in B (by SO of C), so $G' = (G \setminus \{A\}) \cup B$. Since $G \setminus \{A\}$ satisfies SO, B satisfies SO, and no variable of B occurs in G (by renaming apart), G' satisfies SO.
- *Communicate transition*: $G' = G\{X? := T\}$ with $\{X? := T\} \in \sigma$. Since G satisfies SO, $X?$ occurs at most once in G. Replacing that occurrence by T, which satisfies SO (Definition 25), preserves SO provided no variable of T occurs elsewhere in G. A variable occurs in at most one term σ assigns: a clause variable occurs once in its clause, and matching binds the writers of a head to pairwise disjoint subterms of the goal. A variable of an assigned term enters a goal only by the Communicate of that assignment, since a Reduce brings into the goal the variables of a clause renamed apart, which by the proper run condition (Definition 210) are new to the configuration. So no variable of T occurs in G, and G' satisfies SO.

Proposition 212 (Monotonicity) *In any proper cGLP run, if unit goal A can reduce with clause C at step i, then either an instance of A has been reduced by step $j > i$, or an instance of A can still reduce with C at step j.*

Proof. Suppose goal A can reduce with clause C at step i, meaning the writer mgu of A and the head H of (a renaming of) C succeeds. Consider what can change between steps i and $j > i$:

- *Reduce transitions on other goals*: These do not affect A directly. By SO, no other goal shares a writer with A, so no other reduction can assign a writer in A.
- *Communicate transitions*: These instantiate readers, not writers. A communicate transition $X? := T$ may instantiate a reader $X? \in A$, yielding $A' = A\{X? := T\}$. We show A' can still reduce with C:
  The original writer mgu succeeded, meaning at position p where $X?$ occurred in A, either (a) H had a writer Y at position p, yielding assignment $Y := X?$, or (b) H had a reader $Y?$ at position p, which would have caused failure

(reader-reader), or a term, on which the match would have suspended (Definition 213); both contradict success.

In case (a), after the communicate transition, A' has T at position p. The clause C (renamed apart from A') has a fresh writer Y' at position p. The writer mgu now yields $Y' := T$, which succeeds.

– *Reduce transition on A*: If A itself is reduced at some step k with $i < k \leq j$, then an instance of A has been reduced, satisfying the proposition.

Thus, if A has not been reduced by step j, the (possibly instantiated) goal A' at step j can still reduce with a fresh renaming of C.

H GLP Language Primitives and the Root `self.glp`

The scope chain of every module (Section F) starts from the GLP language primitives, followed by the `self.glp` of the root. This section describes both, and the type alias mechanism. The root `self.glp` itself is `programs/self.glp` in the public GLP repository.

H.1 GLP Language Primitives

The GLP language primitives are the primitive types (`Integer`, `Real`, `String`, `Module` and `MutualRef`), the guard predicates, and the body kernels. They are implemented by the runtime and form the base of every module's scope chain; their complete listing appears in the GLP language paper [41]. `Number`, `Constant`, and `Exp` are not primitive: they are defined in the root `self.glp` below. Every scope above them—the root `self.glp` and every application module—is expressed in GLP, introduces no further primitive, and reaches runtime functionality only through the provided primitives.

H.2 Admission to the Primitive Layer

A *system module* is the root `self.glp` or a module under `programs/system/`; it declares the directive `-mode(system)`, and only a system module may define or call a procedure whose name is a `_`-prefixed constant. A constant with that prefix is unrestricted as data—as a message tag, or as a member of a type union. The rule and the directive are defined in the GLP language paper [41]. An application module—one implementing a platform, protocol, or program—declares neither, and reaches runtime functionality by calling the kernel and seam predicates that the root `self.glp` defines [41], as it would any procedure in its scope. The compiler enforces this rather than convention: it rejects a module outside the root `self.glp` and `programs/system/` that declares `-mode(system)`, and a module without `-mode(system)` that defines or calls a procedure so named.

H.3 The Root `self.glp`

The root `self.glp` is the outermost ancestor scope, shadowable like any other, and everything it provides is expressed in GLP.

Among the parameterised type templates it defines are these five. `Stream(X)` is a stream of `X` values, ending in the empty list, and `List(X)` has the same definition under its own name; `OpenStream(X)` is a stream with at least one element, and its continuation is a `Stream(X)`. `DiffList(X)` is a difference list,

for constant-time concatenation. `Channel(In, Out)` is a bidirectional communication channel with read stream `In` and write stream `Out`, the latter consumed by the channel holder and so mode-inverted within the template. Templates are expanded at each use site (Section 6.2).

It defines the derived types `Number`, the union of `Integer` and `Real`, and `Constant`, the union of `Number`, `String` and `Module`; and the arithmetic-expression type `Exp`, the union of `Number` with the arithmetic operators and functions of the catalogue [41], each over `Exp`: the binary `+`, `-`, `*`, `/`, `//`, `mod` and `pow`, and the unary `abs`, `sqrt`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `exp`, `ln`, `log`, `integer`, `real`, `round`, `floor` and `ceil`. The kernel `:=` evaluates an `Exp`, one clause per operator or function (Section 3.2, type checking of `:=`). The empty list is a `String`, hence a `Constant`.

It defines single-unit-clause procedures that serve as defined guards. When called in guard position, the partial evaluator unfolds them at compile time, before type checking. The call `X = T` assigns `T` to the writer `X`. `dl_append` concatenates two difference lists and `dl_to_list` closes one into a stream. `new_channel` produces two channel endpoints, each reading what the other writes. `send` adds a message to a channel's write stream, and `receive` consumes one from a non-empty read stream, its continuation's read stream possibly empty. `close` consumes a closed read stream and closes the write stream. A channel consumer covers its read-stream type with two guarded clauses, one calling `receive` and one `close`: unfolding places the corresponding channel structure in each head, and together the heads accept every input path of the channel (Definition 412).

It also defines the parameterised `merge` procedure of Section 6, over three `Stream(X)` arguments, two consumed and one produced.

H.4 Type Aliases

For readability, programs may define type aliases. A *simple alias* has a single alternative that is a type reference: an alias for a defined type, or for the dual of one. A *union alias* has multiple alternatives, each of which is a type reference; it is expanded by collecting all alternatives from the referenced types. So an alias for the union of a network-message type and a user-message type expands to the alternatives of both. The referenced types must be defined—not aliases themselves, and not primitives—and the expanded alternatives must satisfy the determinism requirement.

During preprocessing, every occurrence of an alias is replaced by its expansion, before type automaton construction. Circular alias chains are rejected.

I Type Automaton Construction

This appendix provides the formal construction of the type automaton summarised in Section 4.2.

Definition I1 (Type Automaton). *Given a typed GLP program $P = (Cs, D)$, the* ***type automaton*** *$\mathcal{A}_D = (Q, \Sigma, \delta, F)$ is defined as follows:*

States Q. *The state set comprises: (1) user-defined type states T and $T?$ for each type name; (2) procedure states p/n; (3) primitive type states (see Appendix H); (4) the states* `Term` *and* `Term?` *(Section 3.1); (5) a final state* ✓.
Alphabet Σ. *Transition labels are tuples (f, n, i, m): functor f, arity n, argument position $i \in \{1, \ldots, n\}$, and mode $m \in \{\uparrow, \downarrow\}$. For constants, the label is $(c, 0, m)$.*
Initial and final states. *The initial state for type T is T; for procedure p/n, it is p/n. Final states are* `_`, `_?`, *and* ✓.

Definition I2 (Transition Function). *The transition function $\delta : Q \times \Sigma \to Q$ is defined by the following rules:*
Procedure transitions. *For a procedure declaration* `p`$(T_1, \ldots, T_n)$*:*

$$\delta(p/n, (p, n, i, m_i)) = \begin{cases} T_i? & \text{if the declaration writes } T_i? \\ T_i & \text{otherwise} \end{cases}$$

The mode m_i is $\downarrow$ for input arguments, $\uparrow$ for output arguments.
Type definition transitions. *For a type definition $T ::= A_1 \,;\, \cdots \,;\, A_k$, each alternative A_j contributes transitions from state T:*
- *If A_j is a constant c: $\delta(T, (c, 0, \uparrow)) = $ ✓*
- *If A_j is a compound term $f(S_1, \ldots, S_n)$: for each $i \in \{1, \ldots, n\}$,*
$$\delta(T, (f, n, i, m_i)) = S_i^{m_i}$$
where $S_i^{m_i}$ denotes S_i if $m_i = \uparrow$, or $S_i?$ if $m_i = \downarrow$ (mode inherited from enclosing context, possibly complemented by embedded ?).
- *If A_j references another type S or $S?$: transitions are inherited from S or $S?$ respectively.*

Primitive type transitions.
- $\delta($`Integer`$, (k, 0, \uparrow)) = $ ✓ *for any integer literal k*
- $\delta($`Real`$, (r, 0, \uparrow)) = $ ✓ *for any real literal r*
- $\delta($`String`$, (s, 0, \uparrow)) = $ ✓ *for any string literal s*
- $\delta($`Module`$, (\mu, 0, \uparrow)) = $ ✓ *for any module term μ*

Dual states have the same transitions with mode $\downarrow$.
The type `Term`. *The state* `Term` *has the transitions of* `Integer`, `Real`, `String` *and* `Module`, *and $\delta($*`Term`$, (f, n, i, \uparrow)) =$ `Term` *for every functor f, arity $n \geq 1$ and position $i \in \{1, \ldots, n\}$.* `Term?` *has the same transitions with mode $\downarrow$, to* `Term?`.
Wildcard states. `_` *accepts any produced term (a writer or ground term with mode $\uparrow$);* `_?` *accepts any consumed term (a reader or ground term with mode $\downarrow$). Both are final states, with no outgoing transitions.*

Definition I3 (Dual Type Automaton). *The automaton for the dual type $T?$ has, for each transition $\delta(T, (f, n, i, m)) = S$, the transition $\delta(T?, (f, n, i, \overline{m})) = S?$, where $\overline{\uparrow} = \downarrow$ and $\overline{\downarrow} = \uparrow$. Complementation is an involution: $(T?)? = T$.*

Example I4 (Type Automaton for Stream) *For the type definition* `Stream ::= [] ; [_|Stream]`*, the type automaton has:*

State `Stream` *(producer view) with transitions:*

– $\delta(\texttt{Stream}, (\texttt{[]}, 0, \uparrow)) = \checkmark$
– $\delta(\texttt{Stream}, (\texttt{"."}, 2, 1, \uparrow)) = \texttt{_}$
– $\delta(\texttt{Stream}, (\texttt{"."}, 2, 2, \uparrow)) = \texttt{Stream}$

State *`Stream?` (consumer view, dual) with transitions:*
– $\delta(\texttt{Stream?}, (\texttt{[]}, 0, \downarrow)) = \checkmark$
– $\delta(\texttt{Stream?}, (\texttt{"."}, 2, 1, \downarrow)) = \texttt{_?}$
– $\delta(\texttt{Stream?}, (\texttt{"."}, 2, 2, \downarrow)) = \texttt{Stream?}$

Example I5 (Type Automaton for Procedure) *For the declaration* `procedure merge(Stream?,Stream?,Stream)`*, the automaton includes:*
State *`merge/3` with transitions:*
– $\delta(\texttt{merge/3}, (\texttt{merge}, 3, 1, \downarrow)) = \texttt{Stream?}$
– $\delta(\texttt{merge/3}, (\texttt{merge}, 3, 2, \downarrow)) = \texttt{Stream?}$
– $\delta(\texttt{merge/3}, (\texttt{merge}, 3, 3, \uparrow)) = \texttt{Stream}$

Path acceptance. A moded path $\pi = (m_0, f_0) \cdot (i_1, m_1, f_1) \cdots (i_k, m_k, f_k)$ determines the label sequence $(f_0, n_0, i_1, m_1) \cdots (f_{k-1}, n_{k-1}, i_k, m_k)$, where n_j is the arity of f_j. The automaton *accepts* a term path π from state q if this sequence leads from q to a state S such that either $S = f_k$, or f_k is a constant and $\delta(S, (f_k, 0, m_k)) = \checkmark$. The type paths $paths(T)$ (Section 4) are the label sequences from T to a primitive or final state: a type path ends at the primitive state and not at a literal, so the literal transitions of a primitive state serve the acceptance of term paths, rows 4–9 of Definition 42, and a clause whose head carries a constant of a primitive type at a consumed position accepts the input path ending there (Definition 410).

J Implementation Notes

This appendix records the implementation decisions behind the type identity of Section F.3, and the checker's inference of a call's instantiation (Section 6.3), and its check of an import against its export (Section F.1). The code is the Dart type-checker of the GLP repository, under `glp_runtime/lib/analysis`.

The instantiation of a call. Definition 62 asks that an instantiation exist and orders nothing; the checker reads the sites of a call over the whole clause, the body goals in no order, and a posted goal, checked as a body (Section F.4), the same way. For each parameter it tries the types the sites supply and the types the callee's clauses fix for it—a head occurrence of the parameter paired by condition 3 with an occurrence of a concrete type, in the body (3(b)) or in the head (3(a))—and takes one under which the clause and the callee's clauses are well-typed with subtyping; where types are supplied or fixed and none serves, the bindings conflict and the call is refused. A parameter for which no type is supplied or fixed is left open where the callee is parametrically well-typed (Section 6.5), the call checked with it open and every argument typed at its position; otherwise the call is refused. A parameter that occurs only as a bare alternative of a type is bound as the check of an import binds it, below.

An import against its export. The check of an imported declaration against the declaration its module exports (Section F.1) finds the instantiation of the

export's parameters by a pairwise walk of the two type automata, not by comparing printed types. An export parameter that occurs only as a bare alternative of a type is bound to the transitions the import's states share at every place it is inlined.

The identity. The type identity of a procedure declaration is the hash of the canonical print of the minimised type automaton reachable from the declaration's state. The construction has four steps. *Expansion* follows the transition function of Appendix I from the declaration's state to the sub-automaton reachable from it, expanding every referenced type and inheriting the transitions of a bare type-name alternative. *Minimisation* is Moore partition refinement; the minimal deterministic automaton of a regular language is unique up to isomorphism, so equal sets of moded paths give isomorphic automata. The *canonical print* is a breadth-first walk from the start state, each state's transitions in sorted label order and states numbered in order of visit, so isomorphic minimised automata print identically. The *hash* is SHA-256 over that print.

One construction for both hashes. The print and the hash are those that build the module source identity of Section F.4. One construction serves both, which is the sense in which the type identity refines the source hash.

Abstracting the procedure name. The reason is in Section F.3; two consequences are operational. Compilation renames every procedure to $M{:}p$ (Section F.2), so an identity carrying the name could never be equal between a goal's author and a shipped module. And the name is fixed on both sides already, by the key under which each table below records the identity.

The tables. The construction produces two tables for a compiled module. The *declared* table holds every procedure declared in the module's scope, the root scope included, and is what `find_type` reads. The *exported* table holds the exported subset, at identical identities, and is the table a `Module` value carries and against which `run` compares. Two further sets of keys are recorded and carry no identity: a parameterised declaration, and a declaration whose signature names a type the scope does not define. The second is empty for a program that type-checks. A key is the procedure's name and arity as the compiled module carries them: the renamed form for a module's own procedures, and the bare form for an entry-point alias.

A version tag. The canonical print opens with a tag naming the version of the construction, which the hash therefore covers. A change to the construction changes the tag, so an identity produced by one version is never equal to one produced by another.

The identity of an opaque instance. A parameterised declaration has no automaton of its own; it has an identity at each opaque instance (Definition F3). The type identity of a type is computed by the construction that serves declarations, from the type's state. An opaque type is a separate state kind carrying its label; in minimisation each label is a class of its own, so two opaque states are merged exactly when their labels are equal, and the print writes the label. A guest's entry counts as the template's instance when it is a declaration of the template's procedure, the template in its ancestor scope, whose automaton

equals the template's under some instantiation, found by the walk of the import check; the guest's exports then record its entry's identity, the identity of the opaque instance, and the list $\iota(\bar{T})$, one identity per parameter in declaration order. `run` admits either recorded identity.

Primitive types. Appendix I gives a primitive type one transition per literal, an infinite family. The print writes in their place one reserved label per primitive per mode, and for the transitions of `Term` on every functor one reserved label per mode. Source symbols are quoted and escaped in the print and reserved labels are not, so a reserved label is never equal to a source symbol.

Deriving the exported table when the artefact is read. A module's artefact carries its interface as text: the reachable type definitions, and the declaration of each export. The exported table will be derived from that text when `load_file` reads the artefact, rather than shipped beside it. A shipped table can disagree with the source it describes; one recomputed from the attested source cannot.

K Related Work

Concurrent Logic Programming. GLP belongs to the family of concurrent logic programming languages that emerged in the 1980s: Concurrent Prolog [38], GHC [47], and PARLOG [11]. These languages interpret goals as concurrent processes communicating through shared logical variables, using committed-choice execution with guarded clauses.

A key evolution was *flattening*: restricting guards to primitive tests only. Flat Concurrent Prolog (FCP) [27] and Flat GHC [47] demonstrated that flat guards suffice for practical parallel programming while simplifying semantics and implementation. Shapiro's survey [39] documents this family and its design space.

GLP can be understood as Flat Concurrent Prolog with the Single-Reader Single-Writer (SRSW) constraint added. FCP introduced read-only annotations (`?`) distinguishing readers from writers of shared variables, enabling dataflow synchronization. However, read-only unification proved semantically problematic: Levi and Palamidessi [24] showed it is order-dependent, and Mierowsky et al. documented non-modularity issues. GHC dispensed with read-only annotations entirely, relying on guard suspension semantics.

Tick [46] surveyed this evolution as a "deevolution" of concurrent logic programming languages, driven by the growing perception that reduced expressiveness is sufficient for practical programming. GLP continues this trajectory: the SRSW constraint—requiring that each variable has exactly one writer and one reader occurrence—resolves the difficulties of read-only unification by ensuring that (*i*) no races occur on variable assignment, and (*ii*) term matching suffices, eschewing unification. The result is a simpler semantic foundation while preserving the expressiveness of FCP, including streams, hollow messages, and meta-programming.

Janus [35] (see also [12]), a concurrent constraint language designed for distributed programming, enforces a two-occurrence restriction on variables—each

variable has exactly one writable occurrence (teller) and one readable occurrence (asker)—which is the constraint-programming counterpart of GLP's SRSW restriction. Lafont's Interaction Nets [23] enforce linearity through graph rewriting: each agent has a single principal port, and interaction rules fire only between agents connected at their principal ports, ensuring deterministic reduction. Both Janus and Interaction Nets share with GLP the design principle that restricting the use of shared resources—whether logic variables, constraint store entries, or graph ports—to single-reader single-writer disciplines yields simpler semantics and enables distributed execution.

Modes in Concurrent Logic Programming. PARLOG used mode declarations at the predicate level, with input modes enforcing one-way matching where clause variables can be instantiated but goal variables cannot be bound.

Ueda's work on moded Flat GHC [49,50] is most directly relevant to Typed GLP. His mode system assigns polarity to every variable occurrence: positive for input/read capability, negative for output/write capability. The *well-modedness* property guarantees each variable is written exactly once—a single-writer constraint matching half of GLP's SRSW requirement. Ueda's subsequent linearity analysis [48] identifies variables read exactly once, enabling compile-time garbage collection for single-reader structures.

GLP enforces both single-reader and single-writer universally as a syntactic restriction, whereas Ueda's system guarantees single-writer with single-reader as an optional refinement. This stronger restriction enables GLP's simpler distributed execution model. Our contribution is a *type system* for this discipline, building on LP types rather than mode inference. To the best of our knowledge, moded types are the first type system for a concurrent logic language whose communication directionality is part of the type structure, obtained as an abstraction of a moded-atom semantics rather than from a separate mode analysis or inference.

Types for Logic Programs. The type system for GLP builds on the Frühwirth *et al.* framework [15], which defines LP types as regular sets of paths and characterises well-typing via path abstraction of the ground-atom semantics. The companion paper [55] establishes the semantic foundations through tuple-distributive sets. Our moded types extend this framework by augmenting paths with mode annotations.

Earlier work on LP types includes Mycroft and O'Keefe's prescriptive polymorphic type system [31] and Mishra's foundational work on tuple-distributivity [29]. Mercury [45] is a practical system separating type checking, mode checking, and determinism inference. Our work differs in integrating modes directly into the type structure rather than treating them as a separate analysis.

Polymorphism by Monomorphisation. GLP's parameterised types and declarations are implemented by *monomorphisation*: each parameterised definition is expanded, and each parameterised procedure checked, once per instantiation the program induces. This places GLP with C++, Rust, Go and Futhark, and with MLton for Standard ML [9,53], against the alternative of a uniform runtime

representation for values of parameter type; Lutze *et al.* [25] state the trade-off as code duplication against the runtime overhead of uniform representations, and present the first general formal treatment of the technique. Two consequences are shared rather than particular to GLP. Expansion needs the whole program, which is why MLton is a whole-program compiler and why the unit of GLP type checking is the program rather than the module. And expansion terminates only up to polymorphic recursion—detected in [25] as cyclic flow in the propagation of type instantiations—which is what GLP's monomorphic-recursion rule and its parameter-subterm restriction on self-referential type definitions each state at their own level.

GLP also has the second route, which monomorphisation does not provide: a procedure that inspects none of its parameters is *parametrically well-typed*, checked once against opaque types and certified for every instantiation (Section 6.5). That is the parametricity discipline rather than the duplication one, and a GLP program may use either per procedure.

Linear Logic, Session Types, and Futures. GLP's SRSW discipline is reminiscent of linear logic's [18] resource sensitivity: each variable assignment is produced at most once and consumed at most once. A GLP reader/writer pair is operationally a future or promise [2,14]: the writer creates a promise for a value, the reader awaits fulfilment. Unlike futures in mainstream languages, GLP futures can carry logical terms containing further reader/writer pairs, enabling recursive communication structures—a capability shared with channel-passing calculi such as the π-calculus [28].

Session types [20,51] use linear logic to type communication protocols. Since GLP messages may contain variables, they allow continuation of the session in both directions, ad infinitum: a reader embedded in a message allows a further message in the same direction, and a writer allows a message in the opposite direction—each of which may contain additional readers and writers. Thus a `Stream` type is a session: each `[X|Xs]` message consists of an element `X` and continuation `Xs`, extending the interaction indefinitely. The `CounterCall` example (Section C.3) is richer: `read(Integer?)` is a message that embeds a writer that inverts the communication direction, enabling the monitor to respond—typed precisely by the mode inversion in the moded type definition. GLP's moded types are thus the logic-programming counterpart of session types, with the mathematical foundations differing in derivation: session types from linear logic sequents, moded types from regular sets of moded paths.

Session type theory distinguishes *duality* from *subtyping*. Duality [20] is an involution $S \mapsto \overline{S}$ that swaps input and output modalities. Subtyping [17] is a preorder for safe substitution: $S <: T$ if a channel of type S can be used where T is expected. Session subtyping is contravariant in input types and covariant in output types.

GLP's moded types incorporate both duality and subtyping directly into the type structure: the $(\cdot)?$ operator relates producer and consumer views of the same channel, analogous to session duality, while subtyping (Section 4.4) allows an interaction that is safe though its types are not exact duals. The well-

typing conditions—containment for outputs, coverage for inputs—embody the same variance principles that session subtyping enforces.

Chen and Pfenning's CoLF$_{\omega_1}$ [10] interprets logic variables as communication channels, compiling mode-checked programs to a concurrent language. Their uniqueness requirement enables stream transducers but precludes availability-driven composition such as fair merge. Systems like SILL$_S$ [3] extend session types with shared state through explicit acquire/release protocols; GLP achieves sharing through merge processes that combine multiple input streams.

Specification-First AI Programming. The development discipline employed in this work—jointly agreeing on types, procedure declarations, and informal descriptions before AI attempts implementation—synthesises classical specification-first methodologies with recent advances in AI-assisted programming. Design by Contract [26] prescribes formal interface specifications (preconditions, postconditions, invariants) before implementation. Literate Programming [22] advocates natural language exposition alongside code. Type-driven development [5] uses types to guide implementation, with typed holes allowing incremental refinement. Recent work extends these ideas to LLM-based code generation: Blinn et al. [4] demonstrate that typed holes provide semantic context for AI code completion, while Mündler et al. [30] show that type constraints during generation reduce compilation errors by over 50%. Our discipline applies these principles at the specification level: types and procedure declarations serve as a contract between human designer and AI programmer, constraining the space of valid implementations before generation begins.